A Survey and Discussion of Memcomputing Machines

Daniel Saunders

July 22, 2016

Abstract

This paper is meant to serve as a review of the recent progress on memcomputing, in which we describe memristors and memristive systems and the memcomputing model of computation. In particular, the *universal memcomputing machine* (UMM) and the *digital memcomputing machine* (DMM) are discussed. We review the memcomputing concept in the dynamical systems framework, assess the algorithms offered for computing NP problems in the UMM and DMM paradigms, and evaluate the arguments for the resolution of the P vs. NP problem.

1 Introduction

The memcomputing model recently introduced by di Ventra, et al. [1] provides a novel way of harnessing nonlinear circuits to perform massively parallel computation. Generally, a memcomputing device can be thought of as a network of memprocessors, simple computing devices which, when wired together, take advantage of their "collective state" in order to compute functions in a massively parallel manner. The device is accompanied by a control unit, which is used to signal computing instructions to the memprocessors. It is important to note that the control unit itself does not perform any computation, in terms of floating-point operations or bit-wise manipulations. After an instruction specified by the control unit has been carried out, the state of the network of memprocessors can be read as the output of the computation step. Memprocessors compute *with* and *in* memory, such that each memprocessor has its own residual memory effects (granted by its memristive makeup) and may interact with up to as many as every other memprocessor in the network. A memcomputing device therefore has no need for a separate CPU, and may save on computation time (due to overcoming the von Neumann bottleneck [8]) and energy expenditure. It is natural to cast the machine as a dynamical system, whose constituent elements are memprocessors, and whose dynamics are derived from the structure of their interconnections and a few simple governing rules.

It is claimed that the digital memcomputing machine is able to solve general NP problems with only polynomial resources (i.e., polynomial time, space, and energy use), boasting identical computational power to that of a *nondeterministic Turing machine* (NTM). It is further argued that there exists a simulation of the digital memcomputing machine (a system of differential equations) implementing an NP algorithm which has only a polynomial blowup in running time, and may therefore resolve the question of P vs. NP in favor of equality. This is supposing, however, the correctness of the algorithms which are said to solve the NP problems, their polynomial worst-case running time, and the correct polynomial time simulation of the operation of the memcomputing machine. This potential result is surprising, as the P vs. NP problem has been open for decades, withstanding a great deal of complexity theory research.

Herein, we reintroduce the memcomputing concept, give a mathematical description of its constituent hardware elements, consider the plausibility of the physical implementations of the machines, and review the arguments for the claims with which it is associated. This paper is meant to be a comprehensive survey of the subject of memcomputing, as well as a skeptical discussion of the complexity theoretic arguments given.

The paper is organized as follows. In section 2, we give a brief review of the state of the art of computation with dynamical systems, in order to gain some insight into current results and open problems. In section 3, we introduce the circuit hardware concept of memristors and memristive elements, first proposed by Chua [10], and later developed by di Ventra, et al. [9]. In section 4, we give an overview of the memcomputing concept and properties of general memcomputing machines. In section 5, we formally define the memprocessor, and show how it can be implemented in nonlinear circuit hardware. In section 6, Universal Memcomputing Machines (UMMs) are introduced, their Turing-completeness is shown, and their application to NP problems is discussed. Section 7 is devoted to the UMM hardware implementation of the $SUBSET-SUM$ problem, and the problems that occur when attempting to scale it to large input sizes. This discussion shows that the UMM concept is difficult, if not impossible, to physically implement. In Section 8, we introduce the Digital Memcomputing Machine (DMM), created to mitigate the problems with the UMM hardware. In section 9, we discuss the proposed implementation of the DMM and the circuit equations that result. Section 10 is devoted to the study of the algorithms for the solution to NP problems, as well as the results in favor of P vs. NP. Finally, Section 11 is devoted to conclusions and potential future work.

2 Computation With Dynamical Systems

We briefly review the notion of computation with dynamical systems. This will allow us to gain some traction on the subject, and to understand what is known about computational devices governed by systems of ordinary differential equations. The hope is that the memcomputing concept will have some bearing on the questions that remain unanswered within this paradigm.

In the landmark work by Siegelmann and Sontag [33], the computational power of analog computation in the context of artificial recurrent neural networks (ARNNs) is studied. It is shown that these ARNNs have super-Turing computation power, namely, the ability to compute all of P/poly in polynomial time, and the ability to recognize arbitrary languages in exponential time. In this work, Siegelmann and Sontag note that it is unlikely that the ARNNs are capable of solving NP-Hard problems in polynomial time, since the $P = NP$ equality implies the almost total collapse of the polynomial time hierarchy within the model. A major contribution of this work is that analog computation is successfully captured by non-uniform models of computation, and indeed, non-uniform circuits of polynomial size can simulate the actions of ARNNs restricted to using polynomial time. The power of this model is a consequence of the real-valued synaptic weights characteristic of the neurons, and, indeed, the model retains Turing computation power when these values are restricted to rational numbers. Further, the idea that "linear precision suffices" is first presented in this work; this means that for a computation time $T(n)$, we require a number of bits of precision proportional to $T(n)$ in the weights of the network in order to guarantee that a given computational problem is solved correctly.

Additionally, Siegelmann and Sontag [36] extended their work on ARNNs to show that one can simulate any multi-stack Turing machine in real time by an ARNN of 886 artificial neurons which computes a universal partial recursive function. Further, it is shown that products (higher order neurons) are not required to

attain Turing-completeness. Finally, the simulation of nondeterministic Turing machines is accomplished by nondeterministic ARNNs.

In Kilian and Siegelmann [38], the computational power of ARNNs with the sigmoid activation function is investigated. These "sigmoidal" networks are useful for the automatic learning of non-linear dynamical behavior. The paper shows that, in the noiseless model, there is a universal architecture which is able to compute any arbitrary recursive function. It is argued that their techniques can be extended to a whole class of "sigmoid-like" activation functions, which shows the robust property of Turing-completeness which recurrent neural network models can obtain.

The computation power of ARNNs augmented with certain discontinuities is studied in [37]. This work shows that adding simple discontinuous functions to an ARNNs whose weights are polynomial-time computable reals allows the networks to compute arbitrarily complex recursive functions, or P/poly in polynomial time. Furthermore, when adding nontrivial period functions such as sine or fractional part, the resulting neural networks are able to compute $PSPACE$ in polynomial time, boasting equal computational power to massively parallel machines.

In [34], Siegelmann describes another model of computation, the analog shift map, a highly chaotic dynamical system which attains full super-Turing computation ability. This model is presented in order to support a generalization of the Church-Turing thesis [19] for analog computation, which is stated as "any realizable analog computer will have no more power (up to polynomial time) than the analog recurrent networks" [34]. Again, the notion that "linear precision suffices" surfaces, which is demonstrated by noting that, if we wish to compute the motion of a particle in a three-dimensional potential up to a certain time q, the number of bits needed to describe the position of the particle and the three-dimensional system is linear in q. This paper provides positive evidence that the analog shift map models the dynamics of general chaotic systems.

In [35], a strict hierarchy of nonuniform computational classes, from Turing to super-Turing, is associated with artificial recurrent neural networks with weights of increasing Kolmogorov complexity. A variant of Kolmogorov complexity is considered, which allows for the consideration of efficiency of the networks in the time domain. In [39], it is shown that real-weighted ARNNs with a real-valued random coin also have super-Turing power (P/poly), and that rational-weighted ARNNs with the same coin are strictly more powerful than rational-weighted ARNNs without, namely, they can compute the complexity class $BPP/\log^*$ in polynomial time. This class is the nonuniform probabilistic class comprised of BPP along with a logarithmic amount of Karp-Lipton advice, and strictly contains the recursive probabilistic class BPP.

The computational power of ARNNs with bi-valued and general plasticity is studied in [40]. It is shown that the four models of ARNNs with either type of plasticity and rational or real weights are equivalently able to compute up to the full super-Turing level. Namely, in order to compute P/poly in polynomial time, the nonuniform polynomial-length advice strings is used as the updates to the synaptic weights of specific neurons over time. This lends credence to the need for computation models which automatically learn and adapt to their environment.

Finally, in [32], Siegelmann explores Alan Turing's ideas on a model of computation which captures biological processes, something that is strictly more powerful than a universal Turing machine. The super-Turing framework is posited as such a model, as a computation device which learns from experiences and adapts to its input, and therefore constitutes an answer to Turing's search for a more robust model of natural

computation.

3 Memristors and Memristive Elements

The first description of the memristor was given by Chua [10] in 1971, then motivated by the gap in the definitions of the relationships between each pair of the four fundamental circuit variables (current, voltage, charge, and flux-linkage). Of the possible six pairings, only the relationship between flux-linkage ϕ and charge q had gone without characterization. It was defined in terms of a new basic circuit element, the memristor, so named for its behavior, which is not unlike that of a nonlinear resistor with memory. At this point in its history, the device was only a theoretical concept, and it was not until 2008 until "the missing memristor was found" by HP Labs scientists [25].

A memristor is defined by the relation $f(q, \phi) = 0$, and is said to be charge-controlled (flux-controlled) if this relation can be defined in terms of a single-valued function of the charge q (flux-linkage ϕ). Though a memristor behaves like a normal resistor at any given instant in time, t_0, its resistance (conductance) depends on the entire past history of current (voltage) across the device. Once the voltage or current over time is specified ($v(t)$, $i(t)$), this circuit element behaviors like a linear time-varying resistor, which effectively describes the memory effects the memristor exhibits. It was shown that the element demonstrates behavior unique from that of standard resistors, capacitors, and inductors, and with these properties there are a number of memristive circuit applications which cannot be recognized with standard RLC networks alone. The nonlinear dynamical behavior of a memristor is characterized by a pinched hysteresis loop, which is what underlies the memory effects seen in memristive behavior.

The new circuit element potentially has profound implications for the construction of new, more efficient circuit hardware at the nanoscale. One exciting feature of certain realizations of the memristor is that it acts as a non-volatile memory; this means that the device retains its internal state (say, resistance or capacitance), even when it is disconnected from power. In this way, the memristor represents a passive and non-volatile memory element which is capable of computation due to the fast switching times of its internal state variable(s).

Di Ventra et al. [9] extended the concept of memristive systems to include capacitors and inductors with memory effects, known as memcapacitors and meminductors. This work gave analogous definitions to the capacitive and inductive elements and described their dynamical properties. Collectively, these elements with memory effects are known as "memelements". Using these elements (in particular, the memristor), Di Ventra, et al [11] describes a new, massively parallel approach to solving mazes. As mazes are prototypical graphs, it is natural to extend this method to solve graph theoretic queries. Indeed, the memcomputing machine approach described in Section 9 is particularly amendable to such problems. Another application of these devices is in simulating neuronal synapses [26]; in particular, when implemented as threshold devices, memristors can learn functions by switching to high or low states of resistance based on the qualitative history of its inputs.

4 The Memcomputing Concept

A major motivation for the adoption of the memcomputing model is the fundamental limitations of the von Neumann architecture. This classic construction specifies a central processing unit (CPU) that is separate from its memory, and therefore suffers a under a constraint known as the *von Neumann bottleneck* [8]; namely, the latency in busing data between the CPU and memory units. The busing action may also waste energy that might otherwise be conserved with a more efficient architecture [8].

Parallel computation was an important step in decreasing the amount of time needed to compute certain problems, but since all parallel processors must communicate with one another to solve a problem, a considerable amount of time and energy is required to transfer information between them [8]. This latency issue thus calls for a fundamentally different architecture, in which data are manipulated and stored on the same physical device. Additionally, parallel computation has its limits; indeed, if $NC \neq P$ (as is widely believed), then there exist problems that are "inherently sequential", which cannot be sped up significantly with parallel algorithms [13]. Furthermore, in building parallel systems, there is a clear fundamental trade-off between the amount of hardware we are willing to operate and the energy cost of operating these devices, a strong physical limitation on our ability to scale parallel computing to large problem instances.

The paradigm of *memcomputing* has recently been introduced as an attempt to answer these problems [1]. Memcomputing machines compute *with* and *in* multi-purposed memory and computation units, thereby avoiding the transfer latency between the main memory and CPU. These machines are built using memristive systems, as explained above, and therefore exhibit time non-locality (memory) in their response functions at particular frequencies of inputs. The fundamental processing elements used by the machines are known as *memprocessors*, which can be composed of the memristors, memcapacitors, and meminductors mentioned in the previous section, though it is possible to build such machines with standard electronic components. However, as the family of memristive components are passive circuit elements, implementing the machines with these devices may save on energy, as the pinched hysteresis loop characterizing memristive behavior implies that no energy needs to be stored during a computation.

We will show later how the memcomputing paradigm can be cast into the dynamical systems framework. For this reason, the algorithm for the solution of problems in NP could be a significant contribution to the study of computation with dynamical systems, as this is a completely new result. For a related study, in [14], Ben-Hur, Siegelmann and Fishman give a model of computation for continuous systems based on ordinary differential equations, whose output is interpreted as the attractors to which the systems converge. This is quite similar to computation within the digital memcomputing machine model, though a memcomputing machine can instead be interpreted as a discrete-time model. These topics are further explored in [15] and [16], where linear programming problems are modeled as systems of ODEs, whose solutions are the attractors to which the system converges, and the complexity of these analog systems are studied, namely, the convergence rate as a function of system size.

A memcomputing machine is composed of a network of memprocessors, otherwise known as the *computational memory*, which is set into motion by a control unit, which is responsible for specifying the functions to be computed by the machine. It is important that the control unit does not perform any computation itself, so that we do not have to bus information between it and the memory. We may provide input data directly to the computational memory, or to the control unit, which feeds in signals carrying computing instructions. The output of the machine can be read from the nodes of the network of memprocessors at the

end of the machine's computation. In particular, in the digital memcomputing machine model, we consider the output of the network the voltages at the terminals which encode the logical 0, 1 bits of the output of the computation.

Now, because of the unique architecture of this class of machines, we can discuss the interesting properties they are said to exhibit. The terminology used below is taken from [1] - [3], among other works by Di Ventra, et al.

4.1 Intrinsic Parallelism

Intrinsic parallelism refers to the feature in which all memprocessors operate *simultaneously* and *collectively* during the machine's computation. This idea is similar (but not identical) to that of the computation of quantum Turing machines (QTMs) over the entangled states of qubits, which can be used to solve certain problems efficiently [13]. Instead of solving problems via a sequential approach as in the Turing paradigm, we can allegedly design memcomputing algorithms which solve problems in fewer steps by taking advantage of the massive parallelism granted by the computation over the collective state of the memprocessors in the computational memory. The idea that this intrinsic parallelism may grant us non-trivial computation speed-up is contested in the computational complexity community [6], but indeed, it is not unlikely that memcomputing machines can grant us constant-factor speedups, especially considering the incredibly fast switching times of the memristive devices being used in their construction.

4.2 Functional Polymorphism

Functional polymorphism is the idea that a memcomputing machine's computational memory can be re-purposed to compute different functions without modifying its underlying topology. This is similar to allowing a Turing machine (TM) access to different programs, or transition functions, in that the signal applied to the current state of the machine determines its operation. The machine's state at any given time, along with the function signal applied to the memprocessors, completely determines the state of the machine at the next computation step. This feature allows the machine to compute different functions by applying the appropriate signals. Thus, the control unit may be fed by a stream of input data, or operate conditionally depending on the state of the computational memory.

We can simulate this behavior on a Turing machine by taking the union of all TMs whose functions we want to compute, and specifying a delegating machine whose job is to execute the respective programs in the correct order, depending on the input and the outputs of the intermediate functions. All machines share the same memory between computations. Note that, in lieu of this observation, the property of functional polymorphism is contained in the Turing paradigm, and therefore does not add anything novel to the theory of computation.

4.3 Information Overhead

Information overhead allegedly allows the machine to store and compress more than a linear amount of data as a function of the number of memprocessors in the network, because of the physical coupling of the individual elements. This amount is strictly larger than that possible by the same number of non-coupled memprocessors, which is characterized by the case of parallel Turing machines (PTMs). We do

not *explicitly* store information into the connections of the computational memory, but rather, we have an *implicit* embedding of information as an emergent property of the computation on the collective state. This information storage is accomplished by the non-volatility of the memristor's internal state variable(s). This concept is reminiscent of "brain-like computation", in which synaptic connections between neurons strengthen or degrade in order to store qualitative information, learned through experience, on the functions they have been trained to compute [21].

5 Memprocessors

We give the general mathematical definition of the memprocessor, which formalizes the relationship between the abstract universal memcomputing model (introduced in the next section) and the possible physical implementations of the machine. It is important to note that these hardware elements can be fabricated from standard electronics, though the passivity property of actual memristive elements will allow us to save on energy expenditure. All definitions and notation are taken from [1].

Definition 4.1: A memprocessor is defined by the four-tuple (x, y, z, σ), where x is the state of the memprocessor (taken from a set of states, which may be a finite set, a continuum, or an infinite set of discrete states), y is an array of internal variables, z is an array of connecting memprocessor elements, and σ is an operator which defines the evolution of a memprocessor, given by

$$\sigma[x, y, z] = (x', y'), \tag{1}$$

where x', y' are the update state and internal variables of the memprocessor, respectively.

When we connect together two or more memprocessors, we have a network of memprocessors, whose state we denote by $\mathbf{x}$, a vector which holds the states of all memprocessors in the network. We denote by $\mathbf{z}$ the union of all connecting variables between memprocessors. Now, if two memprocessors, i and j, are such that $z_i \cap z_j \neq \emptyset$, they are connected. Otherwise, we say that a memprocessor is *isolated* if its connecting variables z are completely determined by its state and internal variables; that is, $z = z(x, y)$. So, in this case, we can write the evolution of memprocessor as

$$\sigma[x, y, z(x, y)] = (x, y). \tag{2}$$

So, an isolated memprocessor has no dynamics.

The evolution of the entire computational memory $\mathbf{z}$ is given by the evolution operator Ξ, which is defined by

$$\Xi[\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{s}] = \mathbf{z}', \tag{3}$$

where $\mathbf{y}$ is the union of all the connecting variables of the network, and $\mathbf{s}$ is the array of signals provided by the control unit at the connection of processors that act as external stimuli. The total evolution of an n-memprocessor network is given by

$$\begin{cases} \sigma[x_1, y_1, z_1] = (x_1', y_1'), \\ ... \\ \sigma[x_n, y_n, z_n] = (x_n', y_n'), \\ \Xi[\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{s}] = \mathbf{z}'. \end{cases}$$

The operators σ and Ξ can be interpreted as either in discrete or continuous time, the discrete time containing the artificial neural network (ANN) model, whereas the continuous-time operators represents a more general class of dynamical systems.

5.1 Memelement Implementation of Memprocessors

The equations governing the behavior of a memelement are given by the following relations [1]:

$$x_{ex}(t) = g(x_{in}, u, t)u(t), \tag{4}$$

$$\dot{x}_{in}(t) = f(x_{in}, u, t), \tag{5}$$

where x_{in} denotes the internal state variable(s) of the memelement, u and x_{ex} are any two complementary constitutive circuit variables (i.e., current, charge, voltage, or flux-linkage) which denote the input and output of the element, g is a generalized response, and f a continuous vector function. In our case, x_{ex} and u coincide with the connection variables z for a single memprocessor, and we write $z = [x_{ex}, u]$, whereas x is captured by the internal state variable(s) x_{in}. Writing the complementary circuit variables u as a function of the internal state and output of the processor, we have $u = u(x_{ex}(t), x_{in}(t))$, a solution of $x_{ex}(t) = g(x_{in}, u, t)u(t)$. Thus, we can replace u in equation (5) by $u(y, x_{in})$, and forget about equation (4). The evolution of a single memprocessor in time T is given by

$$x_{in}(t+T) - x_{in}(t) = \int_t^{t+T} f(x_{in}(\tau), u(x_{in}(\tau), x_{ex}(\tau), \tau))d\tau, \tag{6}$$

and setting $x = x_{in}(t)$, $x' = x_{in}(t+T)$, we have

$$\sigma[x, y, z] = \sigma[x, z] = x_{in}(t) + \int_t^{t+T} f(x_{in}(\tau), u(x_{in}(\tau), x_{ex}(\tau), \tau))d\tau. \tag{7}$$

The operator Ξ is simply defined by Kirchoff's laws and external voltage generators in order to include the signal from the control unit, s.

6 Universal Memcomputing Machines

Herein, we describe the *universal memcomputing machine* (UMM) formulation, compare it with the classic universal Turing machine (UTM), and show that the UMM is Turing-complete. We note here that this universal machine is simply a model of computation, much like the UTM when it was first proposed by A. M. Turing in 1936, and the conjecture is that UMMs are strictly more powerful than UTMs. That is, UMMs are a super-Turing model of computation. However, if we can successfully implement these in hardware, then we have found a counterexample of the physical Church-Turing thesis [17], which states that any reasonable physical model of computation is equivalent in power to the Turing machine. If this is true,

than this contribution is certainly the most significant result of the memcomputing literature, and on this point, we take a skeptical tone. We also give the argument that UMMs are allegedly capable of solving NP problems in polynomial time, taking advantage of the machine's computation over collective states. Because of its analog and therefore inherently noisy implementation, the possibility of a scalable physical realization of a UMM seems dubious.

6.1 Definitions

We will briefly review the definitions of several types of Turing machines and certain relevant complexity classes.

Definition 5.1: Without loss of generality, a *Turing machine* is an abstract computing device comprised of a tape (containing cells which contain a letter from the tape alphabet) used for input, work space, and output, a read-write head that operates over the tape, and a finite set of states in which the machine may be in during its computation. Formally, a Turing machine is defined by the five-tuple

$$TM = (Q, \Sigma, \delta, q_0, F), \tag{8}$$

where Q is the finite set of states, Σ is the set of symbols that may be written on the tape (we use {0, 1, _}, where _ is the blank symbol), q_0 is the initial state of the machine, $F \subseteq Q$ is the set of final states (the states in which the machine will accept if it has halted in one), and Δ is the machine's transition function, which can be formally written as

$$\delta : Q \times \Sigma \to Q \times \Sigma \times \{L, R, N\}, \tag{9}$$

which describes the symbol the machine writes, the state it shifts into, and the left, right, or null movement of the read-write head on seeing a tape symbol $a \in \Sigma$ while in state $q \in Q$.

We say that a Turing machine M decides a language $L \subseteq \{0, 1\}^*$, such that for a string $w \in \{0, 1\}^*$, $w \in L$ if and only if $M(w) = 1$, and $w \notin L$ if and only if $M(w) = 0$, where $M(w)$ denotes the computation of machine M on input string w. For the classic paper on the Turing machine, see [18], and for a more complete treatment, consult the text listed under [13].

Informally, the Church-Turing thesis entails that a Turing machine can compute a function on the natural numbers if and only if a human being can do so, ignoring resource constraints (e.g., time and memory) [19]. In other words, if a problem may be solved via a purely mechanical process (e.g., the reading and writing action of a TM on its tape), or, in other words, is "effectively calculable", it is computable by a Turing machine. Therefore, the Turing machine is a robust model whose computation power is closed under trivial physical modification (such as increasing the size of the tape alphabet or increasing the number of work tapes the machine has access to), save for restricting resources such as time or size of memory.

Definition 5.2: A *universal Turing machine* (UTM) is one whose program, or transition function, is to interpret its input as a tuple encoding a Turing machine description and its input, and to simulate the execution of the encoded Turing machine and input pair on its tape. This is possible with only polynomial slowdown with a k-tape Turing machine (for some $k \in \mathbb{N}$) [13]. Therefore, without loss of generality, we can

consider any particular Turing machine program as one which is simulated on a UTM, without significantly altering the theory being the implementation.

Definition 5.3: A *nondeterministic Turing machine* (NTM) is a Turing machine which has access to a transition function δ which does not have a uniquely specified 3-tuple $(q, a, s) \in Q \times \Sigma \times \{L, R, N\}$ for each $(q, a) \in Q \setminus F \times \Sigma$, and whose operation at each step is determined by an all-powerful "oracle" Turing machine.

Essentially, if there exists a computation on a string w from the initial state of the machine to a halting and accepting state, the oracle machine will choose from among the actions of δ such that the transitions chosen will eventually lead to the halting state. In another realization, the machine branches into all possible paths on each step, each one corresponding to a possible transition (q, a, s). Therefore, assuming that the computation path is polynomial in the length of its input, the solution tree branches into an exponentially growing number of NTMs at each step. A *deterministic Turing machine* (DTM) is a special case of an NTM, which has a unique transition function behavior for every possible state of the machine.

Definition 5.4: P is defined as the class of languages computable in *deterministic polynomial time*, which are those languages L for which there exists a DTM M_L such that $\forall n \in \mathbb{N}$, if $|w| = n$, $M_L(w)$ halts and outputs 1 (if $w \in L$) or 0 (if $w \notin L$), and requires a number of computation steps bounded in $O(n^k)$, for some $k \in \mathbb{N}$.

As a general rule of thumb, P refers to problems which are tractable, or efficiently computable [13].

Definition 5.5: NP is defined as the class of languages computable in *nondeterministic polynomial time*, which are those languages L for which there exists an NTM N_L such that $\forall n \in \mathbb{N}$, if $|w| = n$, $N_L(w)$ halts and outputs 1 (if $w \in L$) or 0 (if $w \notin L$), and requires a number of computation steps bounded in $O(n^k)$, for some $k \in \mathbb{N}$.

An equivalent and perhaps simpler characterization of NP is as the class of languages for which, given a proof, c, that a string $w \in L$, with $L \in NP$, there exists a polynomial-time DTM M_L, know as a *verifier machine*, such that $M_L(w, c) = 1$. Because of this second and equivalent definition, we may think of NP as the class of problems which have *efficiently verifiable solutions* [13].

Definition 5.6: A UMM is defined as the following 8-tuple:

$$UMM = (M, \Delta, P, S, \Sigma, p_0, s_0, F), \tag{10}$$

where M is the set of possible states of a single memprocessor (this may be a finite set, a continuum, or an infinite discrete set of states), Δ is a set of transition functions the control unit commands, formally written as

$$\delta_\alpha : M^{m_\alpha} \backslash F \times P \rightarrow M^{m'_\alpha} \times P^2 \times S, \tag{11}$$

where m_α is the number of memprocessors read by (used as input for) the function δ_α, and m'_α is the number of memprocessors written to (used as output for) the function δ_α. P is the set of arrays of pointers p_α that select the memprocessors called by the function δ_α, S is the set of indexes α, Σ is set of initial states of the memprocessors (written by the control unit at the beginning of the computation), $p_0 \in P$ the initial array of pointers, $s_0 \in S$ the initial index α, and $F \subseteq M$ the set of final states.

This definition is given in [1], and may be too general for our purposes, but we use it in considering the power of this theoretical machine.

6.2 Turing-completeness of UMMs

We show now that UMMs are capable of Turing-complete computation. To do this, we show that a UMM can simulate any single-tape TM, this condition being sufficient for completeness. Consider an arbitrary one-tape Turing machine, M_A.

We take the set of states M to be the union of M_A's state set Q and tape alphabet Σ. We define $p \in P$ as the unique set of pointers, $\{j, j_s\}$, where j_s points the memory cell which denotes the current state of M_A, $q \in Q$, and j as the infinite set which corresponds to the infinite tape of M_A, each pointing to a memory cell encoding a symbol from the tape alphabet. We have that $j = ..., -k, ..., -1, 0, 1, ..., k, ...$, where the zeroth position refers to the tape cell over which the read-write head is positioned. We take Δ to be composed of the transition function of M_A, the function $\delta_{M_A}[\mathbf{x}(p)] = (\mathbf{x}'(p), p')$, where $\mathbf{x}'(p)$ refers to the new symbol of the tape cell which the machine read, as well as the new state of the machine, which we find in $\mathbf{x}'(j_s)$, and $p' = \{j_s, j'\}$, where $j' = j$ if the TM computation step requires no shift on its tape, $j' = j + 1$ if it requires a right shift, and $j' = j - 1$ if it requires a left shift. Now, writing the initial state of M_A into $\mathbf{x}(j_s)$ and writing the machine's input on the appropriate memory cells, the UMM with $\Delta = \delta_{M_A}$ simulates the actions of the Turing machine M_A.

6.3 UMM Computation of NP-Complete Problems

Informally, we define a problem as NP-Complete if it is both in NP and is in some sense one of the "hardest" problems in the class. Using simple polynomial-time reductions from one problem to another, it has been shown that some thousands of natural problems are complete for NP, and because we may reduce any one to another, they are essentially equivalent in terms of computational complexity [13].

Consider an arbitrary NP-Complete problem, and the computation tree of the algorithm implemented to solve it. Since the problem is in NP, the depth of the tree must be some polynomial function of the input length, $p(n)$. At each iteration, we can assume that the computation tree branches into at most a constant number, M, of NTMs, and so the total number of nodes in the tree grows exponentially with the time it takes to traverse a single branch of the tree. This means that a deterministic Turing machine would have to search an exponentially large tree in order to decide a problem instance, and this task takes time exponential in n. On the other hand, it has been conjectured that a UMM may compute this NP-Complete problem in polynomial time. We sketch the argument below.

Consider a UMM with the following scheme: The control unit sends an input signal to a group of memprocessors encoding the ith iteration of the solution tree, to compute the i+1st iteration in one step. The control has access to a set Δ of functions δ, where $\alpha = 1, ..., p(n)$, and $\delta[\mathbf{x}(p_\alpha)] = (\mathbf{x}'(p_{\alpha+1}), \alpha+1, p_{\alpha+1})$,

where $\mathbf{x}(p_i)$ denotes the nodes of the computation belonging to iteration i and $\mathbf{x}'(p_{\alpha+1})$ denotes the nodes of computation belonging to iteration $i+1$. It is important to note that the number of nodes involved at iteration i is at most M^i, and that the final iteration of the algorithm will then involve at most $M^{p(n)}$ nodes.

Now, since each δ_α corresponds to a single constant-time step for the UMM, the solution to the NP problem will be found in time proportional to the depth of the computation tree, $p(n)$. Though this procedure requires only a polynomial number of steps, the necessity for an exponentially growing number of processors causes this approach to be just as implausible as the exponential time algorithm. Indeed, this construction can be accomplished with a exponentially growing number of Turing machines, coupled with a delegating machine to signal the information needed for each iteration. Here, we have a prototypical example of the fundamental parallel time / hardware trade-off characteristic of computational complexity [31].

We further note that these machines have yet to be successfully implemented at scale in hardware, and may prove to be a physical impossibility. In a naive implementation, we require an exponentially growing number of memprocessors in order to store the values on the computation tree of any problem in NP, thereby thwarting the usefulness of the polynomial running time for all practical purposes. However, taking advantage of the property of information overhead, it is claimed that we can reduce this to a polynomially growing amount of hardware. We review this argument in the coming sections.

6.4 Information Overhead Derivation

The following derivation is taken from [1].

To quantify the information stored in a message m, we make use of Shannon's self-information, defined as $I(m) = -\log_2 p(m)$, where $p(m)$ is the probability that the string m is chosen at random from the space of messages, M, and we use the logarithm base two to measure the information in bits. We consider the message space M composed of the set G of n integers with sign, and consider all the possible sums of 2, ..., n integers, taken only once. We define a message m as the integers we are summing and their sum. So, the message space M is composed of $\sum_{j=2}^{n} \binom{n}{j}$ equally probable messages with Shannon's self-information $I(m) = \log_2 \sum_{j=2}^{n} \binom{n}{j}$.

Consider k memprocessors able to store any given integer belonging to the message space M, and we consider the following two scenarios: (i) k independent memprocessors which act as memory cells, and (ii) k memprocessors connected in series. In the first case, we can store at most $\lfloor k/3 \rfloor$ messages, namely, the messages m containing only two numbers and their sum. In the second case, however, by storing k different numbers from the n numbers, we can read out their sum simply by connecting a measurement unit onto the wires which connect each of the k memprocessors we are concerned with. So, in this case, we can store $n(n-1)/2$ messages m, a quadratic number of messages (as a function of the number of memprocessors). So, it is straightforward to see that we can compress a quadratic amount of data simply by connecting the memory cells in series, though measuring the values along the wires requires additional hardware and typically some degree of added computational burden. The quadratic information overhead case is, in fact, equivalent to the scenario in which k independent memprocessors are coupled with a CPU to perform sums over the integers they store.

Now, consider a network of n memprocessors with state variables described by $\mathbf{x}(p) = (\mathbf{x}(j_1), ..., \mathbf{x}(j_n))$. Each memprocessor can store an integer in its internal state, such that $\mathbf{x}(j) = u_j \in M$. We describe the internal state, u_j, as a vector with possibly infinite components $\{u_j\}_h$, and we assume the memprocessor

itself is a "polarized" two-terminal device, in that the left and right terminal, when connected to other circuit elements, interact in physically different ways. We label the left side as "in" and right side as "out" in order to distinguish the two. Furthermore, we assume the blank state of the memprocessor is the situation in which all components of u_j are equal to 0. Now, in order to store a single data point, we set one of the components of u_j different from 0, and assume that any memprocessor can store a finite number of these data points at once. We also assume that there exists a device which, when connected to a memprocessor, can read the components of u_j which it stores.

To complete the construction, we include a scheme of physical interaction between any number of memprocessors. We take two memprocessors, connected via the "out" terminal of the first and the "in" terminal of the second. We assume the read-out device can read the states of both memprocessors at once, i.e., the global state. We describe the global state formally as $u_{j_1,j_2} = u_{j_1} \diamond u_{j_2}$, where $\diamond : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ is a commutative, associative operation, and $d = \dim(u_j)$. It is defined by

$$\{u_{j_1} \diamond u_{j_2}\}_{h \star k} = \{u_{j_1}\}_h * \{u_{j_2}\}_k, \tag{12}$$

where $\star : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$ and $* : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ are two associative, commutative operations such that $\forall h, h \star 0 = h$, $\forall x, x * 0 = 0$. Furthermore, if there are two pairs of indices, (h, k) and (h', k'), such that $h \star k = h' \star k'$, then $\{u_{j_1,j_2}\}_{h \star k} = \{u_{j_1} \diamond u_{j_2}\}_{h \star k} \oplus \{u_{j_1} \diamond u_{j_2}\}_{h' \star k'}$, where $\oplus$ is another commutative, associative operation such that $x \oplus 0 = x$. Since all the operations we have defined are commutative and associative, we can easily iterate these operations to multiple connected memprocessor units.

Now, given a set $G = \{a_1, ..., a_n\}$ composed of n integers with sign, we define a message m as $(a_{\sigma_1} \star ... \star a_{\sigma_k}) \cup \{a_{\sigma_1}, ..., a_{\sigma_k}\}$, where $\{\sigma_1, ..., \sigma_k\}$ is the set of indexes of all possible subsets of $\{1, ..., n\}$, such that the message space M is composed of $\sum_{j=0}^{n} \binom{n}{j} = 2^n$ equally probably messages, each message m having Shannon's self-information $I(m) = \log_2(2^n) = n$. Taking n memprocessors, for each we set only the components u_{j_0} and $u_{j_{a_h}}$ different from zero, and in this way, each memprocessor encodes a distinct element of G. If we were to read the global state of all n memprocessors using the operations defined above, we will find all messages $m \in M$. Since these n interconnected processing units have encoded 2^n bits, we say that we have achieved *exponential* information overhead. It is important to notice that information overhead itself represents the *physical coupling* between memprocessor units, and does not represent the actual transition functions δ_α. While the transition functions change the state of the memprocessors themselves, information overhead is simply related to the *global state* of the memcomputing machine.

We note that this encoding of integers into single memprocessors is impractical, especially for large integers. For this reason, the amount of hardware we require must grow linearly in the *precision* of the integers we are storing, that is, the number of bits needed to represent any given integer. Further, it is not obvious how to design hardware that will read the global state of the machine in one step, especially as the problem instance becomes large (i.e., a large set of integers). We will see in the next section how maintaining an exponentially large collective state quickly becomes intractable.

7 UMM $SUBSET - SUM$ implementation

We review the memcomputing machine implementation described in [2]. For brevity's sake, we will only consider the machine which makes use of exponential information overhead, as the machine that makes

use of quadratic information overhead does not add anything novel in terms of computational complexity. This is because it requires a number of memprocessors which grows exponentially with the size of the set of integers, G, we are considering, and ignores the difficulty in storing integers of varying precision. We discuss the computational complexity of the $SUBSET-SUM$ algorithm, compare it to the complexity in the UMM paradigm as a result of its hardware implementation, and show that this implementation requires only a linear amount of processors, but do indeed require exponential time or energy resources in reading out the result of the computation. For this reason, it is clear that these particular machines are not practical, because of their inability to scale reliably with problem size.

The notation and several arguments below are taken from [1].

7.1 UMM Algorithm for $SUBSET-SUM$

Again, consider a set $G = \{a_1, ..., a_n\}$ of n integers with sign, and the function

$$g(x) = 2^{-n} \prod_{j=1}^{n} (1 + e^{i2\pi f_0 a_j x}). \tag{13}$$

The function $g(x)$ is attenuated by an exponential amount, 2^{-n}, for reasons we will discuss in a later section, and the term f_0 corresponds to the fundamental frequency, equal for every memprocessor. Thus, we are encoding the elements of G as the integer a_j terms. By expanding the above product, it is easily seen that we can store all possible $2^n - 1$ products

$$\prod_{j \in P} e^{i2\pi a_j x} = \exp[i2\pi x \sum_{j \in P} a_j], \tag{14}$$

where P is the set of indexes of all non-empty sets of $\{1, ..., n\}$. So, the function $g(x)$ contains information on all sums of all possible non-empty subsets of G.

Consider the discrete Fourier transform (DFT):

$$F(f_h) = \mathcal{F}\{g(x)\} = \frac{1}{N} \sum_{k=1}^{N} g(x_k) e^{i2\pi f_h x_k}. \tag{15}$$

If the DFT has a sufficient number of points, we will see a peak in correspondence with each f_h, with magnitude which is equal to the number of subsets whose summation is equal to f_h.

It is important to determine the maximum frequency f_{max} such that $F(f > f_{max})$ and $F(f < f_{max})$ are negligible [7]. We define G_+ (G_-) as the subset of positive (negative) elements of G, and it is clear that

$$f_{max} = \max(\sum_{j \in G_+} a_j, -\sum_{j \in G_-} a_j), \tag{16}$$

which may be approximated in excess by

$$f_{max} < n \max(|a_j|). \tag{17}$$

So, $g(x)$ will show peaks which correspond to integer numbers in the range $[-f_{max}, f_{max}]$. Since the elements of G are integers, $g(x)$ is a periodic function of period T which is at most equal to 1. We may then apply the discrete Fourier transform to $g(x)$ which, according to the theory of DFTs and the sampling

theorem [7], will provide the exact spectrum of $g(x)$. From the theory of harmonic balance [7], we can define a number of points

$$N = 2f_{max} + 1, \tag{18}$$

and divide the period T into subintervals of amplitude $\Delta x = N^{-1}$, where

$$x_k = \frac{k}{N}, \quad k = 0, ..., N-1, \tag{19}$$

and then obtain the DFT of $g(x_k)$ using the discrete fast Fourier transform algorithm.

To determine the complexity of the above algorithm, we let n be the size of the set G we are considering, and p the number of binary values required to state the problem. In order to determine $g(x)$ for every x_k we must compute np complex exponential functions and np products of complex variables. We therefore need a total of $4np$ floating point operations.

Now, in order to compute the DFT, we make the following important observation. If we want to pose the $SUBSET-SUM$ problem for only one sum, $f_h = s$, we may use Goertzel's algorithm, which is linear in p. Then, the algorithm for solving the $SUBSET-SUM$ problem for any given s is linear in both n and p. We point out that p is not bounded in n, but rather, depends on N, which grows as an exponential function of the number of bits needed to represent f_{max}.

If we wish to compute the solution for all sums s simultaneously, we can used the FFT which scales with $O(p\log(p))$. So, the total time complexity of this algorithm is $O((n+\log(p))p)$. Note that the best algorithms for large p are dynamic programming solutions, which have complexity of $O(np)$. So, the Fourier transform solution has similar complexity, but has the advantage of a UMM hardware implementation, as we will now discuss.

7.2 Hardware Implementation of $SUBSET-SUM$

First, we connect the concept of exponential information overhead with the algorithm for the subset-sum problem shown above.

Consider memprocessors which are allowed an infinite set of discrete states. In this case, the memprocessors are allowed to take on any integer with sign. Notice that the state vector of a memprocessor u_j can be taken as the vector of the amplitudes of one of the products in (13), and so u_j is such that u_{j_0} and $u_{j_{a_j}} = 1$, with all other components equal to 0. Then the binary operations defined for memprocessors in Section 5.4 are the standard addition and multiplication; in particular, $\star$ and $\oplus$ are sums, and $*$ is a product. These relations show that we can implement the $SUBSET-SUM$ problem using a linear number of memprocessors by making use of exponential information overhead.

Di Ventra, et al [2] then gave the following hardware scheme for implementing the UMM subset-sum algorithm in hardware: Complex voltage multipliers (CVMs) are defined as a two-port device, the first and second ports functioning as the "in" and "out" terminals as described in Section 5.4, respectively. These can be built with standard electronics as seen in [2]. Each CVM is connected to a voltage generator which applies a voltage $v_j = 1 + \cos(\omega_j t)$, where ω_j is the frequency related to an element $a_j \in G$ by the function $w_j = 2\pi a_j$. The state of a single CVM can be read by connecting one port to a DC generator with voltage $V = 1$, and the other port to a signal analyzer implementing the FFT. Now, by connecting the CVMs as

in Figure 11 of [2], and using a signal analyzer on the last port, we can read their collective state, which corresponds to the solution of the subset-sum problem for a given sum s.

To understand the time complexity involved in computing with this hardware, we consider the period T of Equation (13). In the hardware implementation, this completely determines the amount of time necessary to compute the collective state of the CVMs, which is bounded and independent of both n and p. However, in order to evaluate the FFT of the collective state (13), we would in principle need a number of time points on the order of N. This would be troublesome even for a signal analyzer, as N grows in an exponential fashion, thereby requiring us to spend an exponential amount of time reading out the solution. For this reason, before connecting the signal analyzer as depicted in [2], we interpose a band-pass filter which selects only a range of frequencies so that the time samples needed by the analyzer will be bounded and independent of the size of G and the precision of the integers considered. This approach allows the implementation of the subset-sum problem in hardware in a single step, for any integer in $\mathbb{Z}$. Note that in the numerical simulation of this construction, its time complexity is dependent on the number of time points N needed to sample the period T, which implies an exponential time algorithm for $SUBSET - SUM$.

The algorithm given is for the *decision* version of the subset-sum problem, known to be NP-Complete. In order to solve the NP-Hard version of $SUBSET - SUM$, in which we would like to the know the particular subset which sums to a given sum s, the following modifications are made: To find a particular subset which sums to s, we can read the frequency spectrum about s for different configurations of the hardware. In the first configuration, we turn all CVMs on, and in the second, we let $\omega_1 = 0$. If the amplitude corresponding to s is greater than 0 we let $\omega_1 = 0$ for the next configuration, and otherwise turn on ω_1 again, and $a_1 = \omega_1/(2\pi)$ is an element of the subset we are looking for. We then iterate this procedure for all w_i, each corresponding to an element of G, for a number of times which is linear in the size of G, and find one of the subsets which sum to s.

7.3 Problems with the UMM Hardware Implementation

Thanks to [5] and [6], we can present the following arguments against the scalability of the discussed hardware implementation of the $SUBSET - SUM$ problem.

We note two problems in particular. The basic idea of the memcomputing construction is in generating waves at frequencies which encode all the sums of all the possible subsets of G, which we measure in order to decide whether there is a wave with frequency corresponding to a given integer sum s.

The first is a problem with the fact that, in storing the sum of an exponential number of frequencies, we will not be able to distinguish the individual frequencies making up the collective state for an exponential amount of time. Suppose, for example, we are implement a version of the subset-sum construction with $|G| = n = 1000$. Then, we must measure a frequency to a precision of one part in on the order of 2^{1000}. If the fundamental frequency, f, were 1Hz, then the individual frequencies would differ by much less than the Planck scale, and distinguishing between them would require more energy than needed to create a black hole. On the other hand, we can escape this exponential energy blowup by letting the fundamental frequency f be slower than the lifetime of the universe, which would instead cause an exponential blowup in the amount of time required to perform the computation.

On the other hand, since there are an exponential number of frequencies, the magnitude of each wave will be attenuated by an exponential amount. Consider again the case of $|G| = n = 1000$. Then, each

wave is attenuated by a factor of 2^{-1000}. The expected amplitude on each frequency would correspond to a tiny fraction of a single photo. It would then take exponential time to notice any sort of radiation on the frequency which is relevant to the solution of the subset-sum problem. In an ideal machine, it will be possible for us to read the collective state as is presented in [2], but in actual hardware, we are quickly limited by the physical reality of noise.

For this reason, and since no further attempts at a fully analog UMM implementation are presented, it is natural to think that a physical realization of the UMM is difficult, if not impossible, limited by the fundamental physical limitation inherent in storing an exponential number of frequencies or states.

8 Digital Memcomputing Machines

We describe the Digital Memcomputing Machines (DMMs) detailed in [3]. These machines were introduced to bypass the physical limitations of the UMM hardware discussed in the previous section. We first introduce some terminology, taken from [3]:

Definition 7.1: A *Compact Boolean* (CB) *problem* is a collection of statements that can be written as finite system of Boolean functions. Let $f : \mathbb{Z}_2^n \to \mathbb{Z}_2^m$ be a system of Boolean functions, with $\mathbb{Z}_2 = \{0, 1\}$. Then, a CB problem requires that we find a solution $\mathbf{y} \in \mathbb{Z}_2^n$, if it exists, such that $f(\mathbf{y}) = \mathbf{b}$ with $\mathbf{b} \in \mathbb{Z}_2^m$.

Using this terminology, we will later consider the $SUBSET-SUM$ and $FACTORIZATION$ problems, which can be written as finite systems of Boolean functions, for any particular input length.

We consider two different *protocols* with which to solve a CB problem. The former is amendable to a Boolean circuit implementation, while the latter has been developed for DMM computation.

Definition 7.2: Let $S_B = \{g_1, ..., g_k\}$ be a set of k Boolean functions $g_i : \mathbb{Z}_2^{n_i} \to \mathbb{Z}_2^{m_i}$, $S_{CFS} = \{s_1, ..., s_h\}$ a set of h control flow statements, and CF the control flow which specifies the sequence of functions of S_B to be evaluated. Then, the *Direct Protocol* (DP) for solving a CB problem is the control flow CF, which operates by taking $\mathbf{y}' \in \mathbb{Z}_2^{n'}$ as input and returns $\mathbf{y} \in \mathbb{Z}_2^n$ such that $f(\mathbf{y}) = \mathbf{b}$.

The DP is the ordered set of instructions which a Turing machine executes to solve a problem. It is easy to map the finite set of functions into a Boolean circuit, and compute the answer in a feed-forward fashion. For example, consider the $FACTORIZATION$ problem, in which we are given some (possibly composite) number $n \in \mathbb{N}$, and, for the sake of simplicity, are tasked with finding the two prime factors p and q such that $p * q = n$. Given the binary description of n, $n_{binary} \in \mathbb{Z}_2^{n'}$, our CB problem requires that we find p_{binary}, q_{binary}, such that $f(p_{binary}, q_{binary}) = n_{binary}$, where f is simply the multiplication operation. We note that, in this example, the input $\mathbf{y}'$ is equal to the $f(\mathbf{y}) = \mathbf{b}$. It is also worth noticing that the function f is not invertible in this case (nor in many), giving rise to unique solutions only up to ordering ($p * q$, $q * p$), as well as the non-existence of solutions (in the case of prime numbers).

Definition 7.3: The *Inverse Protocol* (IP) is that which finds the solution of a CB problem by encoding the Boolean system f into a machine which accepts as input $\mathbf{b}$, and returns some $\mathbf{y}$ such that $f(\mathbf{y}) = \mathbf{b}$, if

it exists.

So, the IP is an "inversion" of the system of Boolean functions, f. Again, f is not generally invertible, but "special purpose" machines can be designed to solve the invertible instances of CB problems.

The class of DMMs are are strict subset of UMMs, which can be easily seen via its formal definition:

$$DMM = (\mathbb{Z}_2, \Delta, P, S, \Sigma, p_0, s_0, F), \tag{20}$$

where the set of states a memprocessor can take is in $\{0, 1\}$. The key idea is that the memprocessors may only be in one of a finite number of states after the transition functions are applied, but may take on real-valued states during a computation step.

8.1 DMM Implementation of the Inverse Protocol

Using a DMM, we may implement either the direct or the inverse protocol. We describe the latter case, since the first is analogous to the implementation of a Boolean circuit.

Let f be a CB problem given by a finite Boolean system. Since f is composed of Boolean functions, it is straightforward to map it into a Boolean circuit composed of AND, OR, and NOT logic gates. In the same way, f is implemented in a DMM by mapping its functions into the connections of the memprocessors. A DMM may then work in either of the "test" or "solution" modes.

In the test mode, the control unit sends a signal which encodes the input to f, $\mathbf{y}$. The DMM then functions as a Boolean circuit, where the voltages along the input wires propagate in a sequential, feed-forward fashion, and, taking the composition of the transition functions (which correspond to the Boolean functions that have been mapped into the DMM) of the circuit $(\delta_k \circ ... \circ \delta_1)(\mathbf{y})$, the output $f(\mathbf{y})$ is obtained and compared against the expected value of the function, $\mathbf{b}$. The composition of the transition functions $\delta = \delta_k \circ ... \circ \delta_1$ we call the encoding of f into the topology of the connections of the memprocessor network. In Boolean circuit terminology, we can see that each δ_i function corresponds to a layer of the circuit, and so the underlying Boolean circuit has depth k.

In the solution mode, the control unit sends a signal which encodes the output of the function f, $\mathbf{b}$. The first transition function δ_1^{-1} receives it as input, and, taking the composition of the inverse transition functions $(\delta_k^{-1} \circ ... \circ \delta_1^{-1})(\mathbf{b})$, the DMM produces one of the possible inputs (if it exists), $\mathbf{y}$. It is important to note that the Boolean system f is not invertible in general, and so this construction does not prevent the event that the composition of the inverse transition functions, $\delta^{-1} = \delta_k^{-1} \circ ... \circ \delta_1^{-1}$, is not equal to f^{-1}. However, δ^{-1} still represents the encoding of f into the connections of the memprocessors, and for this reason, it is known as the *topological inverse transition function* of f, and we call the system that δ^{-1} represents (some g such that $g(\mathbf{b}) = y$, which may not exist as a Boolean system) the *topological inverse* of f. As we shall later, unlike in the test mode, computation in the solution mode is not accomplished in a feed-forward fashion, but instead the dynamical system describing the DMM's computation will *self-organize*, or converge, to a solution if it exists.

8.2 Comparison With Standard Models of Computation

As we saw above, the DMM functions by encoding a Boolean system f into the connections of its memprocessors. In order to solve an instance of f of a certain length, we must construct a DMM which is

formed by a number of memprocessors and topology which is a function of f and the size of the input. The use of parallelism is especially important in the implementation of the IP used to solve the CB problem, in which the individual memprocessors may interact with many others in the network. We notice that the exact architecture of the DMM may not be unique, since there can be many different topological realizations of the IP for a given CB problem, which is evident in the design of Boolean circuits.

Consider the case of the Turing machine, a device to which we typically provide an ordered set of instructions to in order to perform computation. For this reason, it is obvious that the DP is most amenable to implementation on a Turing machine, although we will see that the simulation of a DMM can be performed on a Turing machine with only polynomial slowdown. DMMs are also quite different from artificial neural networks [21] in that the networks do not fully map the problem to solve into the connections of the neurons. Neural networks are in fact more "general purpose" machines, since they can be trained to solve many types of problems. DMMs, on the contrary, require the correct choice of topology in order to solve a problem.

For this reason, the construction of a DMM for a particular input size is analogous to the construction of a Boolean circuit. For a given problem, we can describe the infinite sequence $D_1, D_2, ...$ where D_i is a DMM which computes the solution to the problem for all inputs of length i. We can in fact take the Boolean circuit family which computes the function f, and use the connections of the logic gates to build the DMM which computes the topological inverse of f, g, as a general-purpose method for solving problems in "reverse". This method is typically only useful for certain types of problems; for instance, NP-Complete problems such as $3SAT$, where we might only care about the existence of a single satisfying assignment to some number of Boolean variables. We may iterate the solution mode procedure to obtain different satisfying results, but in general, there is no way to tell ahead of time what input bits will result from the solution mode computation.

8.3 Information Overhead in DMMs

We must redefine the concept of *information overhead* in terms of the DMM model. We stress that this measure is related to the connections of the memprocessors of the machine, rather than to the data stored in any particular units. The stored information is the Shannon self-information, and is the same for DMMs than that stored by Turing machines. Rather, the information overhead represents the extra data *embedded* into the topology of the connections of the memory units.

Definition 7.1 The *information overhead* is the ratio

$$I_O = \frac{\sum_i (m^U_{\alpha_i} + m'^U_{\alpha_i})}{\sum_j (m^T_{\alpha_j} + m'^T_{\alpha_j})}, \tag{21}$$

where $m^T_{\alpha_j}$ and $m'^T_{\alpha_j}$ are the memprocessors read from and written to, respectively, by the function $\delta^T_{\alpha_j}$ which is the transition function formed by the interconnection of memprocessors with topology related to the CB problem, and $m^U_{\alpha_i}$ and $m'^U_{\alpha_i}$ are the memprocessors read from and written to by the transition function $\delta^U_{\alpha_i}$ which is formed by taking the union of non-connected memprocessors. The sums are over all transition functions used in the computation.

Now, when using the IP, we have that $\delta^T_{\alpha_j} = \delta^{-1}_{\alpha_j}$, defined in the previous section. On the other hand, to employ the DP, we use the transition function of the union of non-connected memprocessors, $\delta^U_{\alpha_i}$.

8.4 Dynamical Systems Perspective

We recast the DMM model of computation as a dynamical system in order to make use of certain mathematical results. This formulation will allow us to give definitions of information overhead and accessible information, as well as to explain the main differences between parallel Turing machines and DMMs. A dynamical systems framework has been used before in order to discuss the computation of artificial recurrent neural networks [21], and, as these are similar to DMMs (we may loosely describe neural networks as memcomputing machines which generally have no particular relation between their topology and the problem being solved), we may extend the theory to DMMs in a natural way.

Consider the transition function δ_α defined previously, with p_α, $p'_\alpha \in P$ as arrays of pointers to certain memprocessors. We describe the state of the network of memprocessors with the state vector $\mathbf{x} \in \mathbb{Z}_2^n$. So, $\mathbf{x}_{p_\alpha} \in \mathbb{Z}_2^{m_\alpha}$ is the vector of the states of the memprocessors selected by the array of pointers p_α. The transition function δ_α then acts as

$$\delta_\alpha(\mathbf{x}_{p_\alpha}) = \mathbf{x}'_{p'_\alpha}. \tag{22}$$

So, δ_α reads the current memprocessor states $\mathbf{x}_{p_\alpha}$ and writes the new states $\mathbf{x}'_{p'_\alpha}$. We note that the transition functions acts simultaneously on all memprocessors, changing their states all at once.

In order to understand the dynamics of the DMM, we define the time interval $\mathcal{I}_\alpha = [t, t + \mathcal{T}_\alpha]$ as the time that δ_α takes to perform its transition. Now, we can describe the computation of the system using the dynamical systems framework [22]. At time t, the control unit sends a signal encoding the transition function to the computational memory, which has state $\mathbf{x}(t)$. The dynamics of the DMM during the interval $\mathcal{I}_\alpha$, between the start and finish of the transition function, is described by the flow ϕ [22]

$$\mathbf{x}(t' \in \mathcal{I}_\alpha) = \phi_{t'-t}(\mathbf{x}(t)). \tag{23}$$

This means that all memprocessors interact at each instant of time in the interval $\mathcal{I}_\alpha$ such that

$$\mathbf{x}_{p_\alpha}(t) = \mathbf{x}_{p_\alpha} \tag{24}$$

$$\mathbf{x}_{p'_\alpha}(t + \mathcal{T}_\alpha) = \mathbf{x}'_{p'_\alpha} \tag{25}$$

We turn to the model of *parallel Turing machines* (PTMs), which we restrict to have at most a polynomially growing amount of hardware (i.e., CPUs) which can perform certain computational tasks in parallel. Because of the hardware restriction, we describe these as "practical" parallel machines (PMs). Indeed, we are interested in a description of the practical approach to parallel computation, which we will compare with that of the DMM model. We assume that all CPUs have access to a shared memory cache, and further that each CPU is synchronized so that each performs a single computation step in time $\mathcal{T}_{PM}$. At the end of each computation step, all CPUs share their results before moving onto the next computation task. For the sake of comparison with the DMM model, we frame these PMs as dynamical systems.

Consider the vector functions $\mathbf{s}(t) = [s_1(t), ..., s_{n_s}(t)]$ and $\mathbf{k}(t) = [k_1(t), ..., k_{n_k}(t)]$, defined respectively as the states of the n_s CPUs and the contents of the n_k memory cells at a given time t. Each CPU performs its computation *independently* of each other at each time step, and thus there are n_s distinct flows which describe their dynamics, of the form $(s_j(t + \mathcal{T}_{PM}), k_{j_\omega}(t + \mathcal{T}_{PM})) = \phi^j_{\mathcal{T}_{PM}}$, where k_{j_ω} is the memory cell written by the j-th CPU. Now, because each CPU reads the memory vector $\mathbf{k}(t)$ only at time step t, and not

in the interval $\mathcal{I}_{PM} = (t, t + \mathcal{T}_{PM}]$, and does not perform any change on $\mathbf{k}(t)$ except to k_{j_ω} at time $t + \mathcal{T}_{PM}$, and so the evolution of the states of the CPUs and memory is completely determined by

$$((s_j(t' \in \mathcal{I}_{PM})), (k_{j_\omega}(t' \in \mathcal{I}_{PM})) = \phi^j_{t'-t}(s_j(t), \mathbf{k}(t)). \tag{26}$$

It is clear from this analysis that the fundamental difference between memcomputing machines and parallel Turing machines is that in each interval $\mathcal{I}_{PM}$ the n_s CPUs do not interact and their dynamics are therefore independent.

Consider a DMM and a PTM, each with the same number, m, of processors (memprocessors and CPUs, respectively), which takes time $\mathcal{T}$ to perform a computation step. We further assume that at times t and $t + \mathcal{T}$, the states of the machine's memory units may only take the values 0 or 1. So the number of possible configurations at these points (initial or final configurations) is 2^m, and the Shannon self-information for both is equal, which is $I_S = -\log_2(2^{-m}) = m$. Consider the following definition, useful as an informal information metric.

Definition 7.2: The *accessible information* I_A is the volume of the configuration space explored by the DMM during its computation (i.e., in the interval $\mathcal{I}_{\mathcal{T}}$).

The concept of accessible information does not satisfy the properties necessary to qualify as a information measure in the formal sense, but the definition does allow for us to highlight interesting features of DMMs which help in pointing out their crucial differences with standard parallel Turing machines. In the case of the PTMs, they are composed of m non-interacting processors, and so from the principals of statistical mechanics [23], the number of total configuration of the machines is the sum of configurations allowed for each element, i.e., $2m$, and so the volume of the configuration space explored by the PTM during the computation is also $2m$. So the accessible information I_A is proportional to $2m$. For the case of the DMM, we have that the m processors interact during the computation, following the dynamics in (23), and so the volume of the configuration space explored by a DMM during the computation is 2^m; i.e., I_A is proportional to 2^m.

Having used the metrics of information overhead and accessible information to discuss the computing behavior of the DMM, we now turn to another important feature of these machines: the composition of their phase space. For a DMM with n memprocessors, the phase space is the n-dimensional space in which $\mathbf{x}(t)$ is a trajectory (which is not to be confused with a configuration space). The extra information embedded in the topology of the DMM (e.g., the encoding of the Boolean system f) strongly affects the dynamics of the DMM.

To link these concepts, we must first note that the dynamical system which characterizes a DMM must satisfy the following properties:

- each component $x_j(t)$ of $\mathbf{x}(t)$ has initial conditions $x_j(0) \in X$, where X is the phase space and is also a metric space.
- For each configuration which belongs to $\mathbb{Z}_2^m$, we can associate one or more equilibrium points $\mathbf{x}_s \in X$, and the system converges exponentially to these equilibria [22].
- The stable equilibria $\mathbf{x}_s \in X$ are associated with the solution(s) of the CB problem encoded in the network topology.

- The input / output (namely $\mathbf{y}$ and $\mathbf{b}$ in test mode or $\mathbf{b}$ and $\mathbf{y}$ in solution mode) of the DMM must be mapped into a set of parameters $\mathbf{p}$ (input) and equilibria $\mathbf{x}_s$ (output) such that $\exists \hat{p}, \hat{x} \in \mathbb{R}$ and $|p_j - \hat{p}| = c_p$ and $|x_{s_j} - \hat{x}| = c_x$ for some $c_p, c_x > 0$ independent of $n_y = \dim(\mathbf{y})$ and $n = \dim(\mathbf{b})$. If we indicate a polynomial function of n of maximum degree γ with $p_\gamma(n)$, then $\dim(\mathbf{p}) = p_{\gamma_p}(n)$ and $\dim(\mathbf{x}_s) = p_{\gamma_x}(n_b)$ in test mode or $\dim(p) = p_{\gamma_p}(n_b)$ and $\dim(\mathbf{x}_s) = p_{\gamma_x}(n)$ in solution mode, with γ_x and γ_p independent of n_b and n.

- Other stable equilibria, periodic orbits, or strange attractors (which we indicate with $\mathbf{x}_w(t) \in X$) which are not associated with solution(s) of the problem may exist, but their exist is either irrelevant or may be ignored by setting initial conditions accordingly.

- It has a *compact global asymptotically stable attractor* [22], which means that there is a compact subset J of X which attracts the whole space.

- The system converges to equilbria exponentially fast starting from an initial configuration whose measure is not zero, and which can decrease at most polynomially with the size of the system. The convergence time may only increase polynomially with the size of the system.

The last conditions implies that the phase space is completely clustered into regions which are the attraction basins of the stable equilibria, and could possibly be periodic orbits or strange attractors, if they exist in the system. We note that the class of systems which have a global attractor is called *dissipative dynamical systems* [22], which will be discussed in a later section. We will show that our DMM construction satisfies the above properties, with the advantage that the only equilibrium points must be solutions to the topology-dependent problem.

9 DMM Implementation

We describe one of the proposed hardware implementations of the digital memcomputing machine architecture. We note that the constituent hardware elements have yet to be built or tested.

In the previous section, we gave the dynamical systems properties which characterize what could be a physical realization of a DMM. From this, it is clear that there is no mathematical limitation in supposing that a system with these properties exists. With the goal of building a physical system that satisfies the requirements of a DMM, we discuss a new type of logic gate proposed by Di Ventra, et al [3] in the hopes of assembling these in a network to form a physical DMM. We can view the logic gates as both a mathematical construct, and as a possible physical implementation.

9.1 Self-Organizing Logic Gates

Recall that a standard n-terminal logic gate is composed of m input and $n - m$ output terminals. The typical logic gate receives signals through the input terminals, performs its logic function over them, and sends the resultant signals to the output terminals. This sequential approach is an implementation of the DP.

On the other hand, we may take any known logic gate (AND, OR, XOR, NOT, ...) and devise a new kind of "self-organizing" gate such that they may work as standard gates (sending input and obtaining an

output), while in "reverse", by selecting an output, the gate will dynamically find an input which is *consistent* with the output, depending on the gate's logic function. We call this new circuit element a *self-organizing logic gate* (SOLG). An SOLG can use all of its terminals simultaneously both as input or output, meaning that signals can go in and out of any terminal at any instant of time, resulting in a superposition of input and output signals. The gate changes the outgoing components of the signals *dynamically*, behaving according to simple rules developed to satisfy its logic relation.

An SOLG may either be in a stable configuration, where its logic relation is satisfied, and the outgoing components of the signal remain constant over time, or an unstable configuration, in which its logic relation is unsatisfied, and so the SOLG drives the outgoing components of the signals in order to obtain stability.

9.2 Self-Organizing Logic Circuits

It is then natural to connect the SOLGs together in a network to obtain a DMM, which may function in either *test* or *solution* mode. So, a *self-organizing logic circuit* (SOLC) is a circuit composed of SOLGs connected with a problem-dependent topology. At each node of the SOLC, i.e., at each interconnection of SOLGs, an external input can be provided, or the output can be read (i.e., the voltage at the node). We note that the topology of the circuit may not be unique, as there are many circuits which realize the same function, and is derived from standard Boolean circuit theory. Therefore, given a CB problem f, it is straightforward to implement f into an SOLC using Boolean relations. The SOLC architecture constitutes a possible realization of the DMM model.

9.3 SOLG Electronics

The SOLGs can be implemented using standard electronic devices. Di Ventra, et al [3] gives a *universal* self-organizing gate which, provided the corresponding signal, can compute either the AND, OR, or XOR function. This is called universal since these logic relations together form a complete Boolean basis set. Choosing a reference voltage v_c, we encode the logic $\{0, 1\}$ as $\{-v_c, v_c\}$, and restrict our SO gates to having fan-in two (v_1, v_2) and fan-out one (v_0). The basic circuit elements used in the SOLC construction are resistors, memristors, and voltage-controlled voltage generators. We discuss the last two in the following.

The standard memristor equations are as follows [1]:

$$v_M(t) = M(x) i_M(t), \tag{27}$$

$$C\dot{v}_M(t) = i_C(t), \tag{28}$$

$$\dot{x}(t) = f_M(x, v_M), \tag{29}$$

$$\dot{x}(t) = f_M(x, i_M), \tag{30}$$

where x denotes the state variables(s) describing the internal state(s) of the system (we assume from now on the existence of only one internal variable); v_M and i_M the voltage and current across the device. M is a monotonic and positive function of x. One possible choice of M is the relation

$$M(x) = R_{on}(1 - x) + R_{off}x, \tag{31}$$

which approximates the operation of a certain type of memristors, where R_{on} and R_{off} correspond to the minimum and maximum resistances of the memristor, respectively. This model includes a small capacitance C in parallel to the memristor which represents parasitic capacitive effects. f_M is a monotonic function of v_M (i_M) for $x \in [0, 1]$ and undefined otherwise. In fact, any function which is monotonic and null for $x \notin [0, 1]$ will suffice to define a memristor. This particular choice of hardware elements is not unique, and can be replaced with other devices; in particular, we can replace the memristor functionality with an appropriate combination of transistors.

The voltage-controlled voltage generate (VCVG) is a linear voltage generator controlled by the voltages v_1, v_2, and v_0. The output voltage is given by a linear combination of these variables of the form

$$a_1 v_1 + a_2 v_2 + a_0 v_0 + dc, \tag{32}$$

whose parameters (a_i's and dc) are defined in Table I of [3], and are chosen to satisfy the relations of the constituent gate (AND, OR, or XOR). We note that these parameters strongly affect the dynamics of the SOLC, and can be summarized as followed:

- If the gate is connected in a network and the configuration of the gate is correct, i.e., the logic relation is satisfied by the incoming voltages, then no current flows from any terminal. We say that the gate is in equilibrium.
- Otherwise, a current of the order v_c/R_{on} flows from the each gate with sign opposite to the voltage impinging on the terminal.

Connecting these gates together in a network, these requirements induce feedback, forcing each gate to eventually satisfy its logic relation, since their correct configurations are stable equilibrium points. We note that since all elements but the VCVGs are passive, the equilibrium points of the gates are both stable and attractive.

With the above SOLC implementation of the DMM, we may not always prevent the existence of stable solutions which are unrelated to the problem to be solved. For this reason, at each terminal except for those acting as input to the SOLC is connected a Voltage-Controlled Differential Current Generator (VCDCG). This device will allow only either v_c or $-v_c$ as stable solutions, thereby eliminating the case of ill-formed stable equilibria. Rather than explaining how it works in detail, we discuss a simplified version of the equations governing the VCDCG as follows:

$$\frac{di}{dt} = f_{DCG}(v) \tag{33}$$

The function has been sketched in Figure 7 of [3]. Considering the voltage v around 0, the above equation can be linearized, giving $\frac{di}{dt} = -m_0 v$, with $m_0 = -\frac{\partial f_{DCG}}{\partial v}|_{v=0} > 0$. This is equivalent to a negative inductor, and is sufficient to make the 0 solution unstable. If we linearize around $\pm v_c$ (the encoding of logic 0, 1), we obtain $\frac{di}{dt} = m_1(v \pm v_c)$, with $m_1 = \frac{\partial f_{DCG}}{\partial v}|_{v=\pm v_c} > 0$. In this case, a VCDCG is equivalent to an inductor in series with a DC voltage generator of magnitude $\pm v_c$. Since these are connected in a network of memristors and voltage-controlled voltage generators, $v = \pm v_c$ are stable points, and any other voltage v induces an increase or decrease in current along the corresponding wire. So, there are no other possible stable points.

9.4 Stability Considerations

Herein, we provide a partial analysis (provided in [3]) of the SOLC circuits equations, and discuss the properties and stability of the circuits as physical systems. Using modified nodal analysis, it has been shown that the SOLC circuit forms a differential algebraic system (DAS) [7]. Since dealing with ordinary different equations (ODEs) is typically simpler, in order to make use of results from functional analysis as applied to dynamical systems theory, an order reduction of the DAS describing the SOLC is performed, and, after reducing the linear part, a system of ODEs is obtained. It is clear that the results found in experimenting with the system of ODEs applies to the original DAS.

Now, having obtained the system of ODEs, the only variables that need to be kept track of are the voltages $v_M \in \mathbb{R}^{n_M}$ across the memristors, with n_M being the number of memristors in the circuit, the internal variables of the memristors $x \in \mathbb{R}^{n_M}$, the currents $i_{DCG} \in \mathbb{R}^{n_{DCG}}$ flowing into the VCDCGs, with n_{DCG} being the number of VCDCGs in the SOLC, and finally, the state variables $s \in \mathbb{R}^{n_{DCG}}$ of the VCDCGs. The system can be formally written as

$$C\frac{d}{dt}v_M = (A_v + B_v\mathcal{D}[g(x)])v_M + A_i i_{DCG} + b, \tag{34}$$

$$\frac{d}{dt}x = -\alpha\mathcal{D}[h(x, v_M)]\mathcal{D}[g(x)]v_M, \tag{35}$$

$$\frac{d}{dt}i_{DCG} = \mathcal{D}[\rho(s)]f_{DCG}(v_M) - \gamma\mathcal{D}[\rho(1-s)]i_{DCG}, \tag{36}$$

$$\frac{d}{dt}s = f_s(i_{DCG}, s), \tag{37}$$

where $\mathcal{D}$ is the linear operator $\mathcal{D}[\cdot] = \text{diag}[\cdot]$. We write the system compactly as

$$\frac{d}{dt}\mathbf{x} = F(\mathbf{x}), \tag{38}$$

where $\mathbf{x} \equiv \{v_M, x, i_{DCG}, s\}$, where F may be read from the RHS of equations (34) - (37). For the sake of brevity, we discuss only the most relevant properties which characterize the behavior of the dynamical system. For the full analysis, refer to [3].

In Equation (34), C refers to the small parasitic capacitance introduced in the memristor equations, and A_v, $B_v \in \mathbb{R}^{n_M \times n_M}$ and $A_i \in \mathbb{R}^{n_M \times n_{DCG}}$ are constant matrices derived from reducing the circuit equations to a system of ODEs [7]. The vector function $g : \mathbb{R}^{n_M} \to \mathbb{R}^{n_M}$ is the conductance vector of the memristors defined component-wise as (compare with Equation (32))

$$g_j(x) = (R_1 x_j + R_{on})^{-1}, \tag{39}$$

where $R_1 = R_{off} - R_{on}$. Finally, the vector b corresponds to a linear transformation of the DC components of the voltage generators. We have the following proposition:

Proposition 8.1: The vector function

$$F_1(\mathbf{x}) = C^{-1}((A_v + B_v\mathcal{D}[g(x)])v_M + A_i i_{DCG} + b) \tag{40}$$

on the RHS of Equation (34), defined on $\mathbf{x} \in \mathbb{R}^{n_M} \times [0,1]^{n_M} \times \mathbb{R}^{2n_{DCG}}$ belongs to $C^\infty(\mathbf{x})$. Further, since the system with no VCDCGs is passive, the eigenvalues $\lambda_j(x)$ of the matrix $A_v + B_v\mathcal{D}[g(x)]$ satisfy

$$\mathrm{Re}(\lambda_j(x)) < 0 \quad \text{for } j = 1, ..., n_M \text{ and } \forall x \in [0,1]^{n_M}. \tag{41}$$

Equation (35) represents the behavior of current-driven memristors with $g(x)v_M$ flowing through the memristor, component-wise. Analogous results can be derived for the case of the voltage-driven memristor, so we confine our discussion to the current-driven case. The coefficient $\alpha > 0$ can be linked to the physics of the memristor [24]. The conductance $g(x)$ has already been discussed. We choose the nonlinear function $h(x, v_M)$ to be

$$h(x, v_M) = (1-e)^{-kx}\tilde{\theta}(v_M) + (1-e)^{-k(1-x)}\tilde{\theta}(-v_M), \tag{42}$$

where $\tilde{\theta}$ is a function which incorporates the threshold effects in the memristor equations [24], and is at least of $C^r(\mathbf{x})$ for some r. We have the following proposition:

Proposition 8.2: Using Equation (42) to represent $h(x, v_M)$, then $h(x, v_M) \in C^\infty(\mathbf{x})$, and for any $x \in [0,1]^{n_M}$ and for any $t > 0$, we have that $\phi_t^x(x) \in [0,1]^{n_M}$, where ϕ_t^x is the flow corresponding to the dynamical system (38) restricted to the variable x. The boundary points 0 and 1 are limit points and for any open ball $B \subset [0,1]^{n_M}$, we have that $\phi_t^x(B) \subset [0,1]^{n_M}$ is an open ball.

Since x should be restricted to $[0,1]^{n_M}$, this result allows us to do so in a natural way by using the function $h(x, v_M) \in C^\infty(\mathbf{x})$. We also have the following result:

Proposition 8.3: Using (42) to represent $h(x, v_M)$, we have that for any particular choice of r, the function

$$F_2(\mathbf{x}) = -\alpha\mathcal{D}[h(x, v_M)]\mathcal{D}[g(x)]v_M \tag{43}$$

on the RHS of Equation (36) defined on $\mathbf{x} \in \mathbb{R}^{n_M} \times [0,1]^{n_M} \times \mathbb{R}^{2n_{DCG}}$ is in the class $C^r(\mathbf{x})$.

For Equation (36), we require that $\gamma > 0$. Each component of the vector function $f_{DCG} : \mathbb{R}^{n_M} \to \mathbb{R}^{n_{DCG}}$ depends on the voltage at the node where the VCDCG is connected. This voltage is expressed as a linear combination of the v_M components

$$v_{DCG_j} = u_j^T v_M + v_{0_j}, \tag{44}$$

where $u_j \in \mathbb{R}^{n_M}$ and $v_{0_j} \in \mathbb{R}$ are constant vectors. We can then write

$$f_{DCG_j}(v_M) = f_{DCG_j}(v_{DCG_j}) = f_{DCG}(v_{DCG_j}). \tag{45}$$

The function f_{DCG} we want to reproduce with the VCDCG is the function depicted in Figure 7 of [3]. This shape can be accomplished by such smooth step functions as arctan or the $\tilde{\theta}$ function discussed above. The function ρ satisfies

$$\rho_j(s) = \rho_j(s_j) = \rho(s_j), \tag{46}$$

and is expressed as

$$\rho(s_j) = \tilde{\theta}^r(\frac{s_j - \frac{1}{2}}{\delta s}), \tag{47}$$

with $0 < \delta s \ll 1$. We have the following proposition:

Proposition 8.4: The vector function

$$F_3(x) = \mathcal{D}[\rho(s)]f_{DCG}(v_M) - \gamma\mathcal{D}[\rho(1-s)]i_{DCG} \tag{48}$$

defined on $\mathbf{x} \in \mathbb{R}^{n_M} \times [0,1]^{n_M} \times \mathbb{R}^{2n_{DCG}}$ is in the class $C^r(\mathbf{x})$.

In Equation (37), we have the function $f_s : \mathbb{R}^{n_{DCG}} \times \mathbb{R}^{n_{DCG}} \to \mathbb{R}^{n_{DCG}}$, defined as

$$f_s(i_{DCG}, s) = -k_s s(s-1)(2s-1) + k_i(1 - \prod_j \tilde{\theta}(\frac{i_{min}^2 - i_{DCG_j}^2}{\delta i}) - \prod_j \tilde{\theta}(\frac{i_{max}^2 - i_{DCG_j}^2}{\delta i})), \tag{49}$$

with k_s, k_i, δi, i_{min}, $i_{max} > 0$ and $i_{min} < i_{max}$. When $k_i = 0$, Equation (49) represents a bistable system. Consider the role of the variable s: when consider only the term including s, we find that it represents a bistable system with stable equilibrium points at $s = 0, 1$, and an unstable equilibrium at $s = 1/2$. We omit the rest of the analysis of this ODE for the sake of brevity, but summarize its behavior as follows: If at least one $|i_{DCG_j}| > |i_{max}|$, then the variable s will approach the unique stable point for $s < 0$, otherwise it will approach the unique stable point for $s > 1$. If at least one $|i_{DCG_j}| > i_{min}$ and all $|i_{DCG_j}| < i_{max}$, then $s \in [\frac{1}{2} - \frac{\sqrt{3}}{3}, \frac{1}{2} + \frac{\sqrt{3}}{3}]$.

From Equation (36) and the definition (47), we have that, if there is at least one $|i_{DCG_j}| > |i_{max}|$, then $s < 0$, implying that $\rho(s) = 0$ and $\rho(1-s) = 1$, causing equation (36) to reduce to $\frac{d}{dt}i_{DCG} = -\lambda i_{DCG}$. So, the currents of i_{DCG} decrease. When, for all j, $|i_{DCG_J}| < |i_{min}|$, s jumps and reaches the stable point $s < 1$, and so $\rho(s) = 1$ and $\rho(1-s) = 0$, causing Equation (37) to reduce to $\frac{d}{dt}i_{DCG} = f_{DCG}(v_M)$. Since this function is bounded, the current i_{DCG} is bounded, and if $k_s >> \max(|f_{DCG}(v_M)|)$, we have that $\sup(|i_{DCG}|) \simeq |i_{max}|$. We conclude with the following proposition:

Proposition 8.5: The vector function

$$F_4(\mathbf{x}) = -k_s s(s-1)(2s-1) + k_i(1 - \prod_j \tilde{\theta}(\frac{i_{min}^2 - i_{DCG_j}^2}{\delta i}) - \prod_j \tilde{\theta}(\frac{i_{max}^2 - i_{DCG_j}^2}{\delta i})) \tag{50}$$

on the RHS of (37) defined on $\mathbf{x} \in \mathbb{R}^{n_M} \times [0,1]^{n_M} \times \mathbb{R}^{2n_{DCG}}$ is at least in $C^r(\mathbf{x})$.

Finally, we conclude with a last proposition:

Proposition 8.6: If the variables $\mathbf{x} \in [0,1]^{n_M}$, $i_{DCG} \in [-i_{max}, i_{max}]^{n_{DCG}}$ and $s \in [s_{min}, s_{max}]^{n_{DCG}}$, then there exist some $v_{max} < \infty$ and $v_{min} > -\infty$ such that $[v_{min}, v_{max}]^{n_M} \subset \mathbb{R}^{n_M}$ is an invariant subset under the flow $\phi_t^{v_M}$ of the system of ODEs (38) restricted to the variable v_M.

9.5 Existence of a Global Attractor

Consider the system of ODEs given in compact form by (39). We define the semigroup $T(t)$ such that $T(t)\mathbf{x} = \mathbf{x} + \int_0^t F(T(t')\mathbf{x})dt'$, or, defining $\mathbf{x}(t) = T(t)\mathbf{x}$, we have

$$T(t)x(0) = x(0) + \int_0^t F(\mathbf{x}(t'))dt'. \tag{51}$$

Now, since we have shown that $F(\mathbf{x}) \in C^r(X)$, where X is the complete and compact metric space $X = [v_{min}, v_{max}]^{n_M} \times [0,1]^{n_M} \times [-i_{max}, i_{max}]^{n_{DCG}} \times [s_{min}, s_{max}]^{n_{DCG}}$, then X is a C^r-semigroup. Moreover, we say that a C^r-semigroup is *asymptotically smooth* if, for any nonempty, closed, bounded set $B \subset X$ for which $T(t)B \subset B$, there is a compact set J which attracts B [27]; here, the definition of "attract" is given as: a set $B \subset X$ is said to *attract* a set $C \subset X$ under $T(t)$ if $\text{dist}(T(t)C, B) \to 0$ as $t \to \infty$ [27]. The following lemma is proved in [3]:

Lemma 8.7: The C^r-semigroup defined by (51) is asymptotically smooth.

The following lemma is a direct result:

Lemma 8.8: The C^r-semigroup defined by (51) is dissipative.

Proof: From the previous lemma, $T(t)$ is asymptotically smooth. From Corollory 3.4.3 of [27], there is a compact set which attracts compact sets of X, and there is a compact set which attracts a neighborhood of each compact set of X. Since X is bounded, the lemma immediately follows.

We now give a few informal definitions in order to prove the existence of a global attractor. A *invariant set* $J \subset X$ is one for which $T(t)J = J$ for all $t \geqslant 0$. A compact invariant set J is called a *maximal compact invariant set* if every compact invariant set of $T(t)$ belongs to J. An invariant set J is *stable* if for any neighborhood V of J, there exists a neighborhood $V' \subseteq V$ of J such that $T(t)V' \subset V'$ for all $t \geqslant 0$. A set J is known as *uniformly asymptotically stable* if J is stable and attracts a neighborhood of J. Finally, a set J is said to be a *global attractor* if J is a maximal compact invariant which attracts each bounded set $B \subset X$. We have the following theorem:

Theorem 8.9: The C^r-semigroup $T(t)$ defined in (51) possesses a uniformly asymptotically stable global attractor A.

Proof: From lemmas 8.5 and 8.6, we have that $T(t)$ is asymptotically smooth and dissipative. Since X is bounded, its orbits are bounded (as orbits of bounded sets must be), and so the theorem follows from theorems 3.4.2 and 3.4.6 of [27].

Roughly speaking, this result means that no matter the choice of initial configuration $\mathbf{x}(0) \in X$, $T(t)\mathbf{x}(0)$ will converge asymptotically to a compact invariant set. Since X here is a compact subset of $\mathbb{R}^{2n_M+2n_{DCG}}$, this set can only contain equilibrium points, periodic orbits, and strange attractors, which are all asymptotically stable [27]. We will see that the system converges exponentially to this compact invariant set, and review the argument for the absence of periodic orbits and strange attractors.

9.6 Equilibrium Points

It can easily be seen from the discussion on Equations (36) and (37) that all equilibria must satisfy $u_j^T c_M + v_{0_j} = -v_c, 0$, or v_c for any j. As seen in Section 8.3, $u_j^T v_M + v_{0_j} = 0$ causes unstable equilibria, while the other cases lead to asymptotically stable equilibria. Together with additional stability requirements laid out in [3], these are the necessary conditions to have equilibrium points for $T(t)$. This does not, however, guarantee that $i_{DCG} = 0$ at equilibrium.

For our purposes, we need $i_{DCG} = 0$ for all components j. At the equilibria, the SOLC has voltages at gate nodes that can only be $\pm v_c$. In such a configuration, the gates can be in a correct or incorrect configuration (as seen in Section 8.3). In the first case, no current flows from the terminals due to the behavior of the SOLGs, and so the corresponding component of i_{DCG} must be 0 in equilibrium. In the second case, if the gate configuration is incorrect, at equilibrium we have currents on the order of v_c/R_{on} which flows from the gate terminals with sign opposite to its incoming voltage. These currents will correspond with the components of i_{DCG}. Indicating the minimum absolute value of the current flowing from the gate terminals when in the wrong configuration by $K_{wrong} v_c/R_{on}$, $K_{wrong} = O(1)$, and consider VCDCGs with

$$i_{DCG_{max}} < K_{wrong} v_c/R_{on}. \tag{52}$$

Then, the equilibria with nonzero components of i_{DCG} disappear, and only equilibria such that $i_{DCG} = 0$ can persist. We now have the following theorem:

Theorem 8.10: If the condition in Equation (52) holds, then the uniformly asymptotically stable equilibria for T(t), if they exist, satisfy

$$i_{DCG_j} = 0, \tag{53}$$

$$s_j = s_{max}, \tag{54}$$

$$|u_j^T v_M + v_{0_j}| = v_c, \tag{55}$$

for any $j = 1, ..., n_{DCG}$. These conditions imply that all gate relations must be satisfied at once.

In fact, this theorem is equivalent to the statement $T(t)$ *has equilibria if and only if the CB problem implemented in the SOLC has solutions for the given input instance.*

In [3], exponential convergence to these equilibria, if they exist, is proved. We omit this here, but note the following theorem:

Theorem 8.11: Polynomially growing SOLCs support at most polynomially growing time constants.

This is claimed to be derived from the discussion on the exponential convergence of the dynamical system governing the SOLC's behavior, and is not formally proved in [3]. Indeed, the existence of limit cycles or strange attractors unrelated to the problem could cause the dynamical system governing the computation of the SOLC to oscillate forever. Additionally, the memprocessor network is highly ordered, in the sense that there is a non-trivial periodicity in the construction of the SOLC (namely, the repetitive structure of the Boolean circuit which is "inverted" to form the SOLC). For this reason, we can study the network from

its minimal cell, (i.e., the minimal sub-network that can be iterated in order to construct the entire network being analyzed) in order to analyze the time complexity involved in the computation of the SOLC.

9.7 Absence of Periodic Orbits and Strange Attractors

We have seen previously that $T(t)$ has a uniformly asymptotically stable global attractor. We have also seen the result that there is exponential convergence to the equilibria of this system, and that this convergence rate grows at most polynomially with the size of the system. We will now address the composition of the global attractor, in order to complete the analysis of the DMM model. Again, this analysis is taken from [3], to which the reader should refer to for a complete treatment.

To analyze the composition of the global attractor, we take a statistical approach, and make the following assumptions:

- The capacitance C is small enough so that if we perturb a potential in a node of the network, the perturbation propagates in a time $\tau_C \ll \tau_M(\alpha)$, where $\tau_M(\alpha)$ is the switching time of the memristors. For the SOLC system, the time constant τ_C is related to the memristance and C.
- In the function f_{DCG} (see [3]), q is small enough such that the time $\tau_{DCG}(q) = i_{DCG_{max}}/q$ satisfies $\tau^{-1} \ll \tau_{DCG}$. In other words, the time that the current i_{DCG} takes to reach $i_{DCG_{min}}$ is much smaller than the time to reach $i_{DCG_{max}}$.
- The switching time of the memristors satisfies $\tau^{-1} \ll \tau_M(\alpha) \ll \tau_{DCG}$.
- The initial configuration of the system $\mathbf{x}$ is chosen randomly in X.

We note a peculiar behavior of the SOLCs which can be proved by the nonlocality due to Kirchoff's laws and how the SOLG hardware functions. If we change a potential of the order δV in a switching time $\tau_C \ll \tau \ll \tau_M(\alpha)$, there will be a subnetwork including the point switched, S, such that in a time on the order of τ_C, will change of the order of δV potentials at the nodes of the network. After a time of approximately $\tau_M(\alpha)$, S will reach a stable configuration as a consequence of the SOLG dynamics. So for a perturbation of about δV at a single node of the circuit, there is a reconfiguration of an entire region S of the network in time $\tau_M(\alpha)$.

We can then make a statistical interpretation of the SOLC network. We consider a large enough system and a small enough time τ_C such that the density of the nodes in S is uniform. Taking a random initial configuration $\mathbf{x}(0)$ in X, we have that, at the rate of τ_{DCG}^{-1}, a fraction $\frac{m_{DCG}}{n_{DCG}}$ of VCDCGs reaches i_{max} and then switches. We conclude that there is a fraction of nodes that are "kicked" by $\delta V \approx K_{wrong} v_c / R_{on}$ at a rate of τ_{DCG}^{-1}.

In this way, if $\mathbf{x}$- is the configuration of the SOLC before a "kick", and $\mathbf{x}$+ after, their temporal distance being of the order of γ^{-1}, then the distance dist($\mathbf{x}$-, $\mathbf{x}$+) is of the order of the radius of X, which is defined as $\inf_{y \in X} \sup_{x \in X} \mathrm{dist}(x, y)$. Therefore, these kicks bring the system to points of X which are far from each other. Since we have taken the initial condition of $\mathbf{x}$ randomly in X, the kicks will then take place randomly in time and space, allowing the system to explore the entire phase space X. It is claimed in [3] that the SOLC explores X in a fashion that is linear in the number of variables needed to represent the dynamical system; it needs an amount of kicks that is linear in the dimension of the system.

We can conclude that strange attractors and periodic orbits cannot coexist with equilibria in the SOLC system. Both the periodic orbits and strange attractors produce large fluctuations of the potentials at the nodes of the SOLC. These are on the order of v_c/R_{on}, and are distributed in the entire network due to the physical nature of the VCDCGs and SOLG hardware. Therefore, if periodic orbits and/or strange attractors exist, they will force the system to explore the whole phase space X. So if an equilibrium point does exist, then by exploring the space, the system will intercept, within a time on the order of τ_{DCG} multiplied by a number which grows linearly with the dimension of $\mathbf{x}$, the stable manifold of X, and will therefore collapse into the equilibrium point. So, the global attractor can only be formed either by equilibrium points *or* periodic orbits and/or strange attractors.

9.8 Polynomial Energy Expenditure

Finally, we note that the SOLCs grow polynomially with the input size, each node of each SOLG can support only finite voltages (cut off by the parasitic capacitance C), and at the equilibrium, the voltages do not depend on the size of the system. The currents are also bounded and are independent of the size of the SOLC. If the solution is found in polynomial time, then the energy expenditure can also be described by a polynomial function.

10 DMM Implementation of NP Problems

In this section we provide a discussion of the SOLC for solving the NP-Hard problem $FACTORIZATION$, and explain how this construction can be accomplished for general problems in NP. We review the argument for the SOLC implementation with only polynomial resource expenditure.

Without loss of generality, consider $n = pq$, where p and q are prime numbers. Let p_j, q_j, n_j be the j-th binary coefficient such that $p = \sum_{j=0}^{n_p} p_j 2^j$, and similarly for q and n, and n_p equal to the number of bits required to represent p, and similarly for q and n. The $FACTORIZATION$ problem requires that we find p, q such that $pq = n$. Iterating the procedure for integers n with more than two factors is straightforward.

In order to cast this as a compact Boolean problem, f, we may write the product of two numbers in binary using a closed set of Boolean functions. Thus, we can write the product as $f(\mathbf{y}) = \mathbf{b}$, where $\mathbf{b}$ is the integer n in binary, and $\mathbf{y}$ is the concatenation of the bits of the primes p, q. We note that f is only unique up to the ordering of factors.

10.1 Resource Expenditure

Now, we give the argument that this CB problem can be solved in the IP using only polynomial resources, i.e., circuit hardware, time, and energy expenditure. As an example, the circuit computing the factorization of integers of 6 bits is sketched in [3]. The inputs of the SOLC are given by voltage generators giving the critical voltage v_c encoding the bits of the integer n. We say that the collection of voltage generators is the control unit of the DMM.

The wires of the circuit at the same potential, indicated as p_j, q_j, are the solutions to the problem once the SOLC has self-organized, encoding the bits of the prime factors p and q. To read the output of the SOLC, it is sufficient to measure the voltage along these wires, and because the values of the potentials on all wires can only be $\pm v_c$ after self-organization, there is no problem with precision in reading the output.

This is a departure from the UMM hardware implementation, where noise was an important difficulty when attempting to scale.

As for the space complexity of the circuit, the size of the SOLC for $FACTORIZATION$ grows quadratically with the length of the integer n. In fact, the construction relies on the Boolean circuit for integer multiplication, and, performing an "inversion" on this multiplication circuit and using the SOLGs and associated hardware, we obtain the SOLC used for implementing the factoring algorithm in the IP. From this, it is clear that we can build an SOLC for $FACTORIZATION$ for any $|n|$. Based on the analysis in the previous section, the circuit converges exponentially to the equilibria (which are the prime integer solutions to the problem), in a time supposedly polynomial in $|n|$. Since the energy usage of the circuit depends linearly on the time it takes to find the equilibria and the number of gates in the circuit, if the running time is polynomial, then the energy expenditure is also bounded by a polynomial function.

Now, in order to simulate the system of ODEs which govern the behavior of the circuit, we need a bounded time increment dt which is independent of the size of the circuit, and dependent only on the fastest time constant governing its function. This depends on the nature of the hardware which we are simulating. Now, if there exists a solution to the prime factorization problem, i.e., if the input is a composite number, and the SOLC fulfills the dynamical systems requirements of Section 7.4, then the problem can allegedly be solved in hardware using only polynomial resources. This last claim has serious implications for computational complexity theory, and for this reason, we will argue in a later section for the implausibility of this result.

If the integer we are considering is a prime number, then there is no solution to the factorization problem, and the SOLC will never be in equilibrium. This is also true for the case in which the number of bits of the factors (n_q, n_p) used to build the circuit is smaller than the actual length of the factors, $|p|$, $|q|$. In order to avoid this last case, we choose $n_p = n_n - 1$ and $n_q = \lfloor n_n/2 \rfloor$ (or the reverse), which guarantees that if p and q are prime integers, the solution is unique, and that the trivial solution $n = n \times 1$ is impossible.

10.2 Numerical Simulations

In [3], the SOLCs for integer factorization were implemented into the NOSTOS (NOnlinear circuit and SysTem Orbit Stability) simulator [28]. For the sake of simplicity, the SOLCs were restricting to having outputs of equal length, i.e., $|p| = |q|$. Circuits of several input sizes were built in this fashion, and the simulations were performed by starting at a random initial configuration of the internal variables $\mathbf{x}$ and gradually switching on the voltage generators. After a short time, all terminals approach the critical voltage $\pm v_c$, encoding the logical 0 and 1. When they have converged to these stable points, they are necessarily satisfying all gate relations at once, and so the SOLC has found a solution to the $FACTORIZATION$ problem.

Di Ventra, et al [3] have performed hundreds of simulations in this manner, and have gotten positive results for this problem for integers of length up to 18 bits. This 18-bit case requires the simulation of a dynamical system with on the order of 11,000 variables (the numerical values of $\{v_M, x, i_{DCG}, s\}$), and so we are dealing with an enormous phase space where only equilibria are to be found. This does not prove the absence of unrelated limit cycles or strange attractors for all sizes. Indeed, we cannot say anything definite about the correctness of this dynamical systems algorithm's behavior asymptotically, since we have only empirical evidence of its correct behavior for inputs of at most 18 bits.

On the other hand, when given a prime number as input, the trajectories of the SOLC do not find

equilibria. Since we know that the dynamical system governing the circuit has a global attractor, these trajectories must be characteristic of some complex limit cycle or strange attractor.

In [3], a similar SOLC construction is given for the solution to the $SUBSET - SUM$ problem, but for the sake of brevity, we omit this from the current work.

10.3 Discussion on P vs. NP

We first remark that the problems of $FACTORIZATION$ and $SUBSET - SUM$ were solved in [3] by mapping these NP problems into the NP-Complete problem SAT (i.e., the language of Boolean formulas in conjunctive normal form for which there exists an assignment to its variables, x_1, ..., x_n, such that the formula evaluate to TRUE), and subsequently mapping the SAT formula into the connections of the SOLC. In the same way, we can take any problem in NP and, after performing its polynomial-time reduction to SAT (which is possible by the fact that SAT is complete for NP [13]), we can map the problem into the connections of an SOLC. So, we have a general-purpose framework for implementing NP problems within the DMM paradigm.

We will now review the arguments given in [3] for the evidence for the positive resolution of $P = NP$. The first result that is relevant to this problem are the constraints given in Section 7.4, which constitute a DMM able to solve NP problems using only polynomial resources. It is worth noting that these constraints together are self-consistent. The conclusion is that such a physical dynamical system can exist, though building it for a certain problem instance length may take exponential resources. However, the polynomial running time of the circuit is questionable, as the absence of limit cycles or strange attractors has not been proved, and their presence may cause the running time to grow at an super-polynomial rate, after having to "kick" the system some number of times related to the exponential number of configurations the DMM's SOLGs could be in. However, it is claimed in [3] that the system requires only a number of kicks linear in the size of the circuit.

The time resources needed to simulate SOLCs can be quantified by the number of floating-point operations need to solve the system of ODEs given by (38). Using a forward integration method (such as Euler or Runge-Kutta [29]), the number of floating-point operations needed depends *linearly* on the dimension of the system variables, $\mathbf{x}$, and also depends on the minimum time increment, dt, used to integrate with respect to the total computation time; in other words, this depends linearly on the number of time steps N_t.

We have seen in Section 9.1 that $\dim(\mathbf{x})$ (proportional to the number of variables needed to simulate the system) scales polynomially with the size of the input (quadratically in the number of bits for the integer factorization), and, to put it more simply, depends *linearly* on the number of logic gates (AND, OR, XOR) needed to encode the problem's corresponding SAT instance.

The number of time steps, N_t, needed to complete the computation has a double bound. The first depends on the minimum time increment of the SOLC, dt, and the second on the minimum period of simulation T_s. The first is independent of the size of the SOLC, and depends only on the shortest time constant of the circuit. This means that this depends on the nature of the hardware devices we are simulating. On the other hand, T_s can depend on the size of the dynamical system which represents the SOLC. It is the minimum time we need to find the equilibria, which is related to the largest time constant of the circuit. We gave the argument for the polynomial growth of this time constant in Section 8.8, but it seems possible that the larger time constants of this dynamical system grow exponentially quickly, as the system must exhibit exponential

sensitivity to initial conditions.

The overarching claim is that, from the polynomial time scaling of the numerical simulations given in [3] and the absence of strange attractors and periodic orbits, this is strong evidence for the correctness of the dynamical systems algorithm. However, as these simulations have only been shown to work for inputs of at most 18 bits, this is meager empirical evidence when considering the 1024-bit keys which govern the security of the RSA encryption scheme. Indeed, many NP researchers have been led astray by the promise of polynomial scaling algorithms for relatively small problem instances which turn out to require exponential time in the limit. So, the evidence for the polynomial time memcomputing solution of problems in NP should be taken with a grain of salt, and we should expect that these will require exponential time in the worst case, i.e., for certain "hard" instances of NP problems. See [30] for an extensive list of P vs. NP attempts for more information.

11 Conclusions and Future Work

We have surveyed the memcomputing model of computation, including the definitions of both the universal and digital models, their hardware implementations, and their physical plausibility with scale. Though the dynamical systems picture and computation over the collective state are interesting contributions, it is clear from the DMM discussion that this can be numerically simulated on a Turing machine without a significant amount of overhead (a polynomial slowdown). Implementing these machines in hardware, however, may afford important constant-factor speedups and energy savings because of the fast switching times and passivity property of the memristor hardware.

The question of P vs. NP remains unanswered without proofs of the correctness and polynomial running time of the $FACTORIZATION$ or $SUBSET-SUM$ algorithms, and the relative ease with which we can describe the "collective state" computation is negative evidence in and of itself of the implausibility that this has proved $P = NP$. Namely, since neither quantum or analog computation is explicitly being harnessed here (unless it has been "swept under the rug", as with the analog behavior of the memristor hardware), and because the general consensus is that the question will resolve as $P \neq NP$ [13], there is little reason to believe that these machines have solved the long-standing problem. The most immediate contribution here is perhaps the dynamical systems algorithm for the solution to NP problems, which, if correct, at the very least seems to be a promising SAT solver, and, at best, a platform for which to continue investigation into the question of P vs. NP.

The most pressing issue is the proof of the absence of limit cycles and strange attractors unrelated to the solutions of the problem being solved. Further, a detailed analysis of the worst-case running time should help in settling the question of how this framework bears on the question of P vs. NP. It is rather straightforward to implement the DMM as a SAT solver, and experimenting with these on difficult instances of the problem could shed light on the dynamical behavior of the SOLC circuits in the limit. Implementing a DMM as a "general purpose" machine is desirable, since building new hardware for each problem instance length will be expensive. The development of new dynamical systems algorithms or extensions of the DMM construction will be crucial in understanding how certain classes of problems can be solved within the dynamical systems framework, and in this regard, there is much work to be done.

12 Acknowledgments

We thank professors Fabio L. Traversa and Massimiliano Di Ventra for helpful discussion and clarifications, and professors Hava T. Siegelmann, David A. Mix Barrington, and Scott Aaronson for their invaluable insights.

References

[1] Traversa, Fabio Lorenzo, and Massimiliano Di Ventra. "Universal Memcomputing Machines." IEEE Trans. Neural Netw. Learning Syst. IEEE Transactions on Neural Networks and Learning Systems 26.11 (2015): 2702-715. Web.

[2] Traversa, F. L., C. Ramella, F. Bonani, and M. Di Ventra. "Memcomputing NP-complete Problems in Polynomial Time Using Polynomial Resources and Collective States." Science Advances 1.6 (2015)

[3] Traversa, F. L. and Massimilian Di Ventra. "Polynomial-time solution of prime factorization and NP-hard problems with digital memcomputing machines." 2016 Design, Automation & Test in Europe Conference & Exhibition (DATE), Dresden, 14-18 March 2016. Dresden: IEEE

[4] M. Di Ventra and Y. V. Pershin, "The parallel approach,"Nature Physics,vol. 9, pp. 200–202, 2013.

[5] Markov, Igor L. "A Review of 'Mem-computing NP-complete Problems in Polynomial Time Using Polynomial Resources" Cornell University Library. Cornel University Library, 22 Apr. 2015. Web. 09 May 2016.

[6] Aaronson, Scott. "Memrefuting." Web log post. Shtetl-Optimized. N.p., 11 Feb. 2015. Web.

[7] F. Bonani, F. Cappelluti, S. D. Guerrieri, and F. L. Traversa, Wiley Encyclopedia of Electrical and Electronics Engineering, ch. Harmonic Balance Simulation and Analysis. John Wiley and Sons, Inc., 2014.

[8] J. Backus, "Can programming be liberated from the von neumann style?: A functional style and its algebra of programs," Commun. ACM, vol. 21, pp. 613–641, Aug. 1978.

[9] M. Di Ventra, Y. Pershin, and L. Chua, "Circuit elements with memory: Memristors, memcapacitors, and meminductors," Proceedings of the IEEE, vol. 97, pp. 1717–1724, Oct 2009

[10] Chua, L. "Memristor-The Missing Circuit Element." IEEE Trans. Circuit Theory IEEE Transactions on Circuit Theory 18.5 (1971): 507-19. Web.

[11] Pershin, Yuriy, Yuriy Pershin, and Massimiliano Di Ventra. "Solving Mazes with Memristors: A Massively-parallel Approach." Nature Precedings (2011): n. pag. Web.

[12] Cook, Stephen A. "The Complexity of Theorem-proving Procedures." Proceedings of the Third Annual ACM Symposium on Theory of Computing - STOC '71 (1971): n. pag. Web.

[13] Arora, Sanjeev, and Boaz Barak. Computational Complexity: A Modern Approach. Cambridge: Cambridge U, 2010. Print.

[14] Ben-Hur, Asa, Hava T. Siegelmann, and Shmuel Fishman. “A Theory of Complexity for Continuous Time Systems.” Journal of Complexity 18.1 (2002): 51-86. Web.

[15] Ben-Hur, Asa, Joshua Feinberg, Shmuel Fishman, and Hava T. Siegelmann. “Probabilistic Analysis of a Differential Equation for Linear Programming.” Journal of Complexity 19.4 (2003): 474-510. Web.

[16] Ben-Hur, Asa, Joshua Feinberg, Shmuel Fishman, and Hava T. Siegelmann. “Random Matrix Theory for the Analysis of the Performance of an Analog Computer: A Scaling Theory.” Physics Letters A 323.3-4 (2004): 204-09. Web.

[17] Piccinini, Gualtiero. “The Modest Physical Church-Turing Thesis.” Physical Computation A Mechanistic Account (2015): 263-73. Web.

[18] Turing, A. M. “On Computable Numbers, with an Application to the Entscheidungsproblem.” Annual Review in Automatic Programming (1960): 230-64. Web.

[19] Copeland, Jack. “The Church-Turing Thesis.” NeuroQuantology 2.2 (2007): n. pag. Web.

[20] G. Goertzel, “An algorithm for the evaluation of finite trigonometric series,” The American Mathematical Monthly, vol. 65, no. 1, pp. 34–35, 1958.

[21] H. T. Siegelmann, Neural networks and analog computation: beyond the Turing limit. Springer, 1999.

[22] L. Perko, Differential equations and dynamical systems, vol. 7. Springer Science & Business Media, 3nd ed., 2001.

[23] L. Landau and E. Lifshitz, Statistical Physics. Elsevier Science, 2013.

[24] M. Di Ventra and Y. V. Pershin, “On the physical properties of memristive, memcapacitive and meminductive systems,” Nanotechnology, vol. 24, p. 255201, 2013.

[25] Strukov, Dmitri B., Gregory S. Snider, Duncan R. Stewart, and R. Stanley Williams. ”The Missing Memristor Found.” Nature 459.7250 (2009): 1154. Web.

[26] Jo, Sung Hyun, Ting Chang, Idongesit Ebong, Bhavitavya B. Bhadviya, Pinaki Mazumder, and Wei Lu. ”Nanoscale Memristor Device as Synapse in Neuromorphic Systems.” Nano Letters Nano Lett. 10.4 (2010): 1297-301. Web.

[27] J. Hale, Asymptotic Behavior of Dissipative Systems, vol. 25 of Mathematical Surveys and Monographs. Providence, Rhode Island: American Mathematical Society, 2nd ed., 2010.

[28] F. L. Traversa and F. Bonani, “Improved harmonic balance implementation of floquet analysis for nonlinear circuit simulation,” AEU - Inter. J. Elec. and Comm., vol. 66, no. 5, pp. 357 – 363, 2012.

[29] J. Stoer and R. Bulirsch, Introduction to numerical analysis. SpringerVerlag, 2002

[30] Aaronson, Scott. “The P-versus-NP Page.” P-versus-NP Page. N.p., n.d. Web. 09 May 2016.

[31] Dymond, Patrick W., and Stephen A. Cook. ”Complexity Theory of Parallel Time and Hardware.” Information and Computation 80.3 (1989): 205-26. Web.

[32] Siegelmann, Hava T. "Turing on Super-Turing and Adaptivity." Turing on Super-Turing and Adaptivity. N.p., n.d. Web. 22 July 2016.

[33] Siegelmann, Hava T., and Eduardo D. Sontag. "Analog Computation via Neural Networks." Theoretical Computer Science 131 (1994): 331-60. Elsevier. Web.

[34] Siegelmann, Hava T. "Computation Beyond the Turing Limit." NCBI. Science, 28 Apr. 1995. Web.

[35] Balcazar, J.l., R. Gavalda, and H.t. Siegelmann. "Computational Power of Neural Networks: A Characterization in Terms of Kolmogorov Complexity." IEEE Trans. Inform. Theory IEEE Transactions on Information Theory 43.4 (1997): 1175-183. Web.

[36] Siegelmann, Hava T., and Eduardo D. Sontag. "On the Computational Power of Neural Nets." Proceedings of the Fifth Annual Workshop on Computational Learning Theory - COLT '92 (1992): n. pag. Web.

[37] Gavaldà, Ricard, and Hava T. Siegelmann. "Discontinuities in Recurrent Neural Networks." Neural Computation 11.3 (1999): 715-45. Web.

[38] Kilian, Joe, and Hava T. Siegelmann. "The Dynamic Universality of Sigmoidal Neural Networks." Information and Computation 128.1 (1996): 48-56. Web.

[39] Siegelmann, Hava T. "Stochastic Analog Networks and Computational Complexity." Journal of Complexity 15.4 (1999): 451-75. Web.

[40] Cabessa, Jérémie, and Hava T. Siegelmann. "The Super-Turing Computational Power Of Plastic Recurrent Neural Networks." Int. J. Neur. Syst. International Journal of Neural Systems 24.08 (2014): 1450029. Web.